Operation

강의주소: https://www.inflearn.com/course/iOS-Concurrency-GCD-Operation#

목차 (2부) - Operation

1. Operation에 앞서

1) GCD와 Operation의 차이

2) 언제 Operation(오퍼레이션)이 필요할까?

3) 들어가기에 앞서

2. Operation / OperationQueue

1) 오퍼레이션(Operation)

2) OperationQueue (오퍼레이션큐)

3) BlockOperation (블락오퍼레이션)

3. AsyncOperation (비동기 오퍼레이션)

4. 오퍼레이션큐의 주요 기능

1) 순서 관리 (종속성 - Dependencies)

2) 오퍼레이션큐의 작업 취소

5. 오퍼레이션 프로젝트 살펴보기

6. 총정리 - GCD / Operation의 비교와 사용

1. Operation에 앞서

---

1) GCD와 Operation의 차이

2) 언제 Operation(오퍼레이션)이 필요할까?

3) 들어가기에 앞서

1) GCD와 Operation의 차이

GCD

- 간단한 일(커뮤니케이션의 양)
- 함수를 사용하는 작업 (메서드 위주)

/

기반

(+ 여러가지 기능)

Operation

- 복잡한 일(커뮤니케이션의 양)
- 데이터와 기능을 캡슐화한 객체

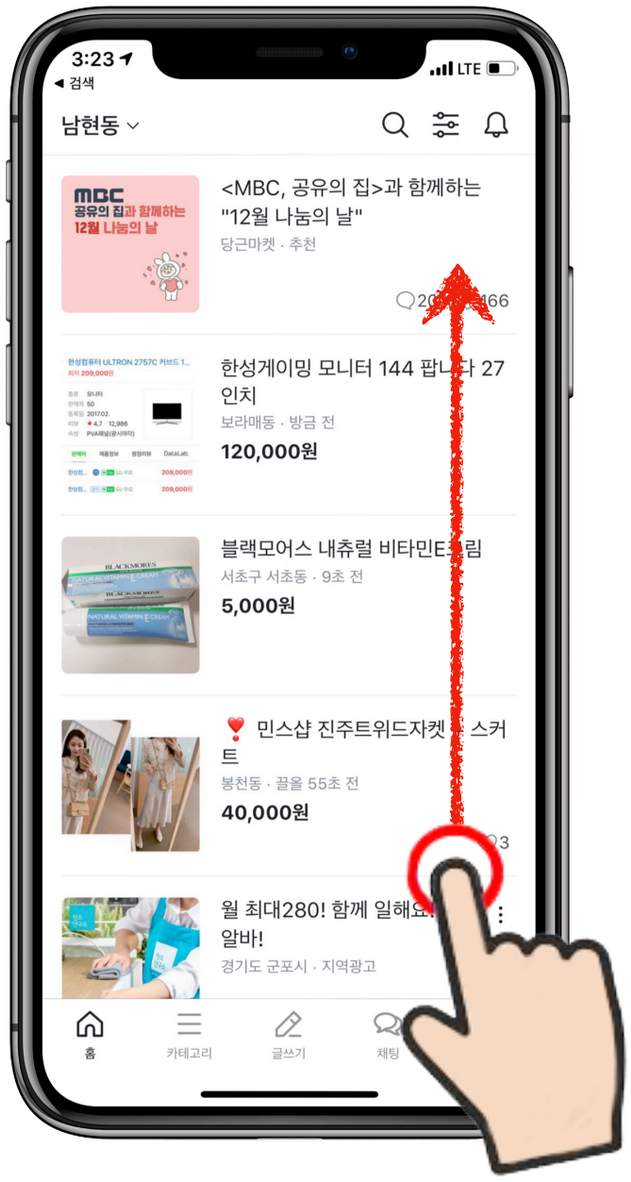
취소 / 순서지정 / 일시중지 (상태추적)

프로젝트의 효율성, 사례 적합성 등

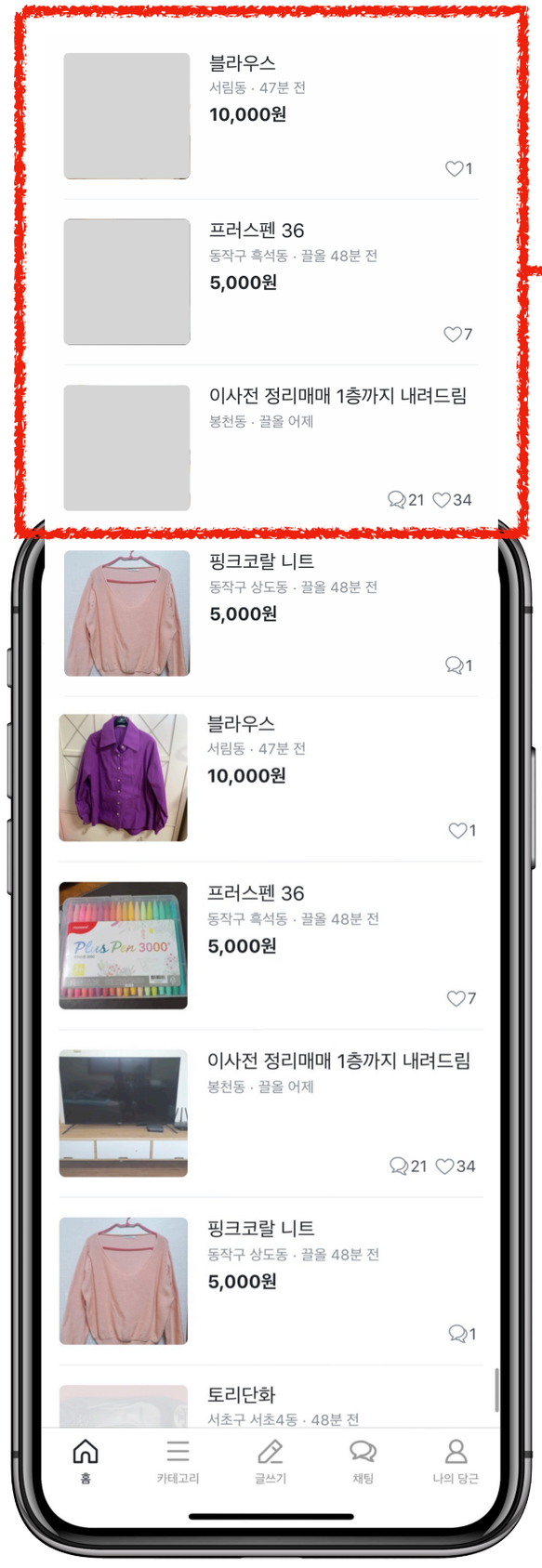
강의주소: https://www.inflearn.com/course/iOS-Concurrency-GCD-Operation#

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

2) 언제 Operation(오퍼레이션)이 필요할까?



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com



이미 지나간 영역

이미지를 표시할 필요가 있을까?

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

3) 들어가기에 앞서

2. Operation / OperationQueue

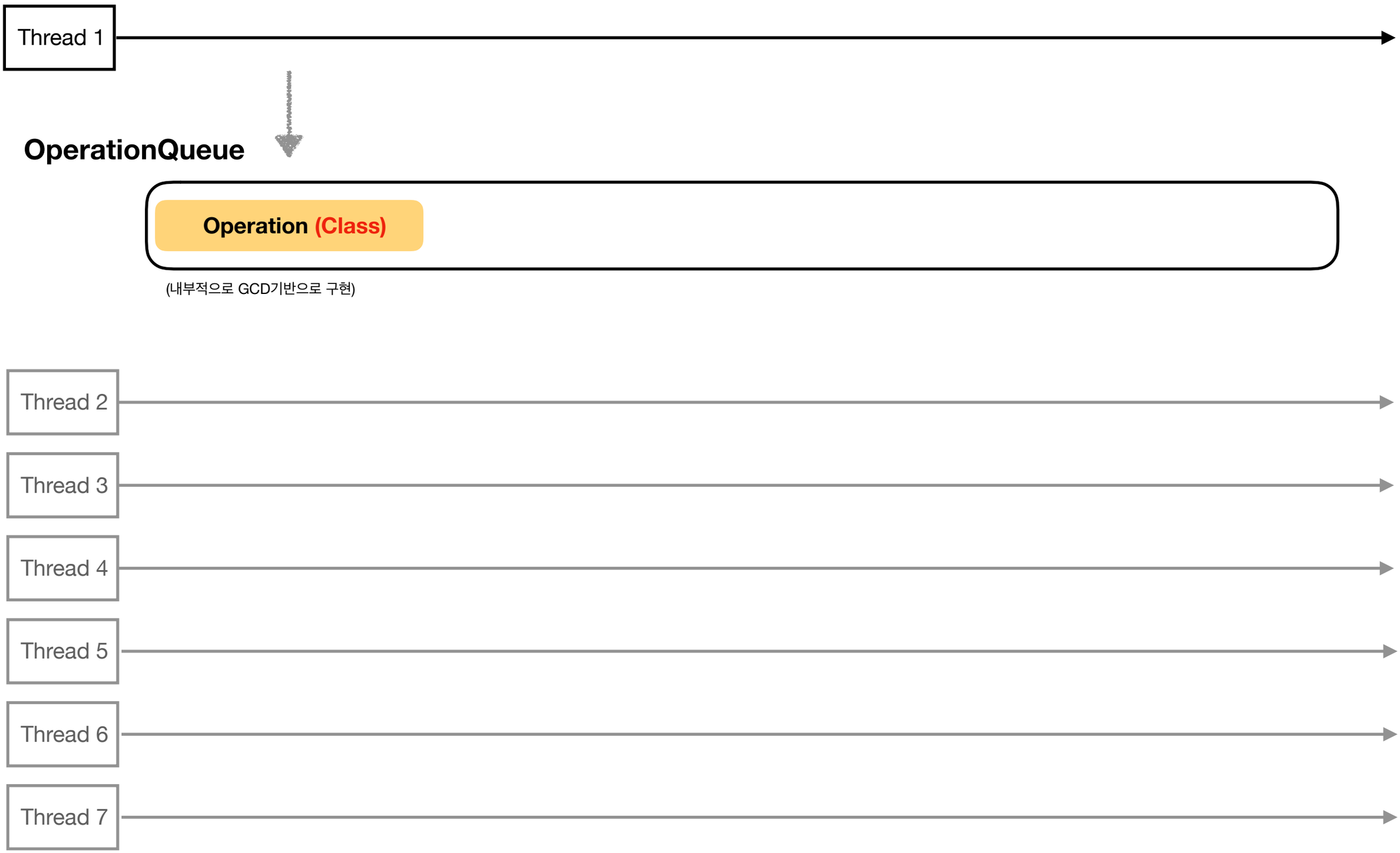
1) 오퍼레이션(Operation)

2) OperationQueue (오퍼레이션큐)

3) BlockOperation (블락오퍼레이션)

1) 오퍼레이션(Operation)

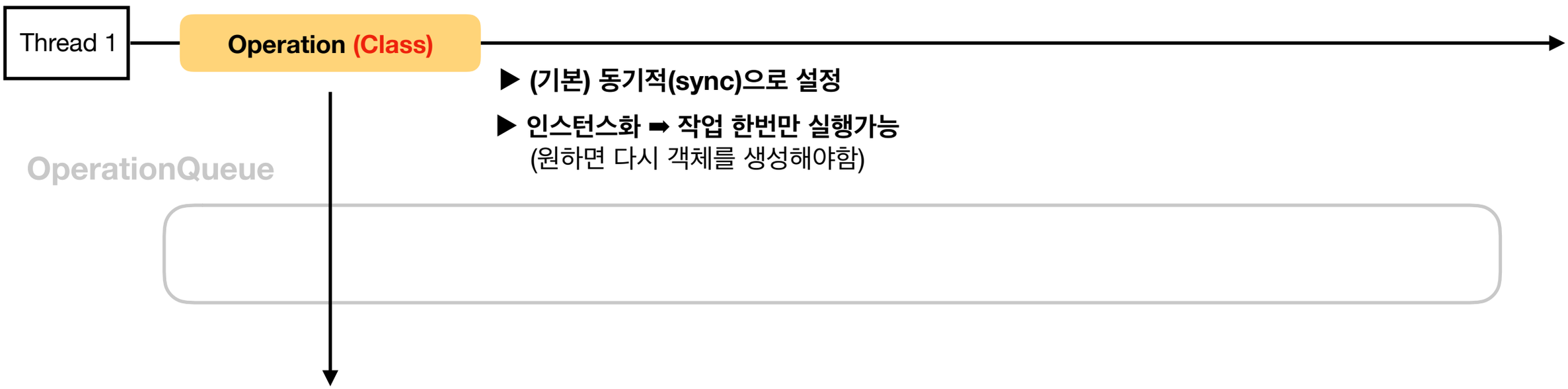
Operation 기본개념



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Operation 기본개념

단위(패키지) 작업



어떤 단위적인 작업을 클래스화해서 기능을 Up + “오퍼레이션큐”와 결합해서 “비동기적으로” 많은 추가 기능들을 사용할 수 있음

(작업 패키지)

(예) 데이터 다운로드

압축풀기

이미지 필터 적용하기

(재사용성)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Operation 기본개념

단위(패키지) 작업

Thread 1 — **Operation (Class)**

▶ **(기본) 동기적(sync)으로 설정**

▶ **인스턴스화 ➡ 작업 한번만 실행가능**
(원하면 다시 객체를 생성해야함)

OperationQueue

어떤 단위적인 작업을 클래스화해서 기능을 Up
(작업 패키지)

(예) 데이터 다운로드

압축풀기

이미지 필터 적용하기

(재사용성)

Operation의 고유기능

취소

순서지정(의존성)

상태 체크(state machine)

KVO notifications

Qos수준

completionBlock 제공

Operation의 메서드 및 변수

▶ **start( ) 메서드**
(오퍼레이션큐에 안넣고, 독립적으로
동작 가능하지만 일반적으로는 오퍼레이션큐 사용)

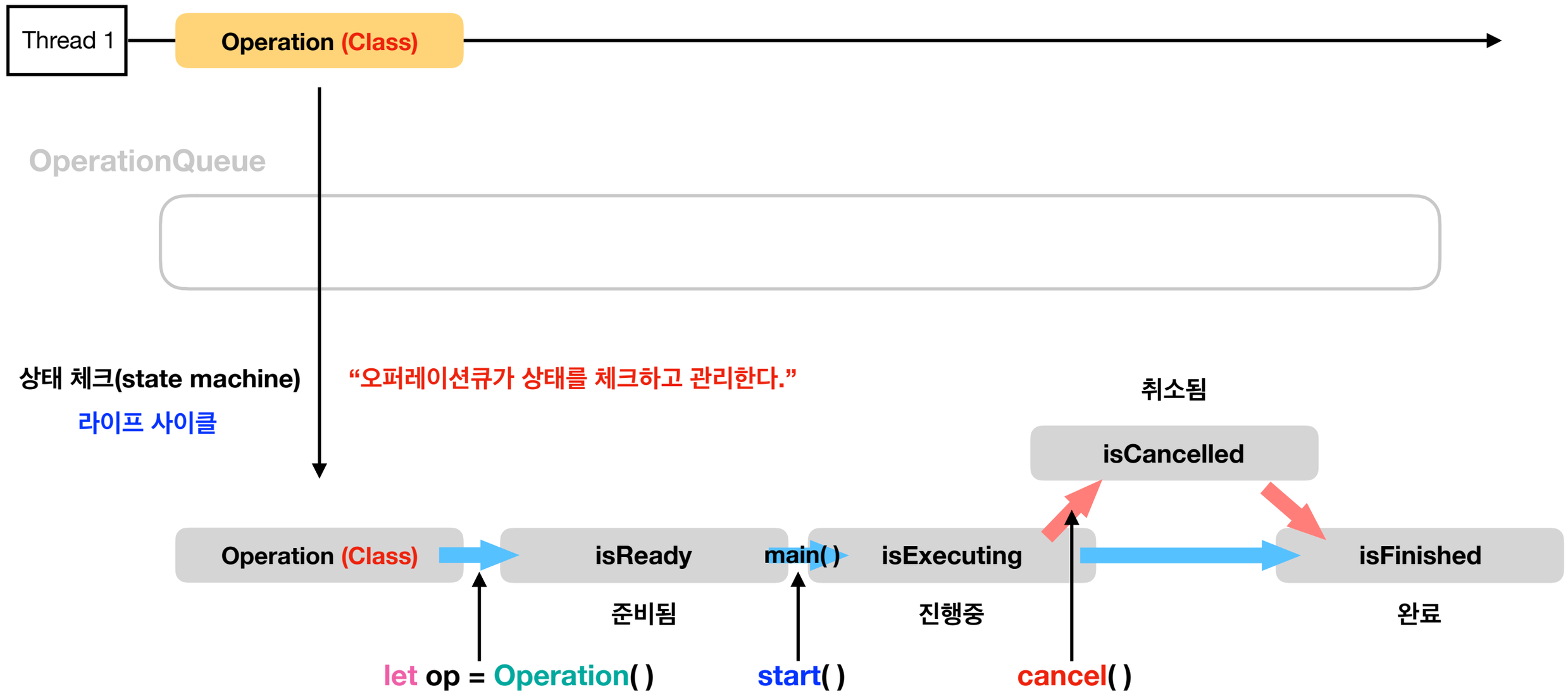
▶ **cancel( ) 메서드**

▶ **상태 체크 위한 Bool값 변수**

isReady
isExecuting
isCancelled
isFinished

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Operation 기본개념



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Operation 기본개념

Thread 1

Operation (Class)

OperationQueue

isReady	true
isExecuting	false
isCancelled	true
isFinished	false

취소됨

상태 체크(state machine)

라이프 사이클

"오퍼레이션큐가 상태를 체크하고 관리한다."

➡ 궁극적으로 순서지정 / 취소하기 위함

isCancelled

Operation (Class) → isReady → main( ) → isExecuting → isFinished

let op = Operation( )

준비됨

start( )

진행중

cancel( )

완료

isReady	false
isExecuting	false
isCancelled	false
isFinished	false

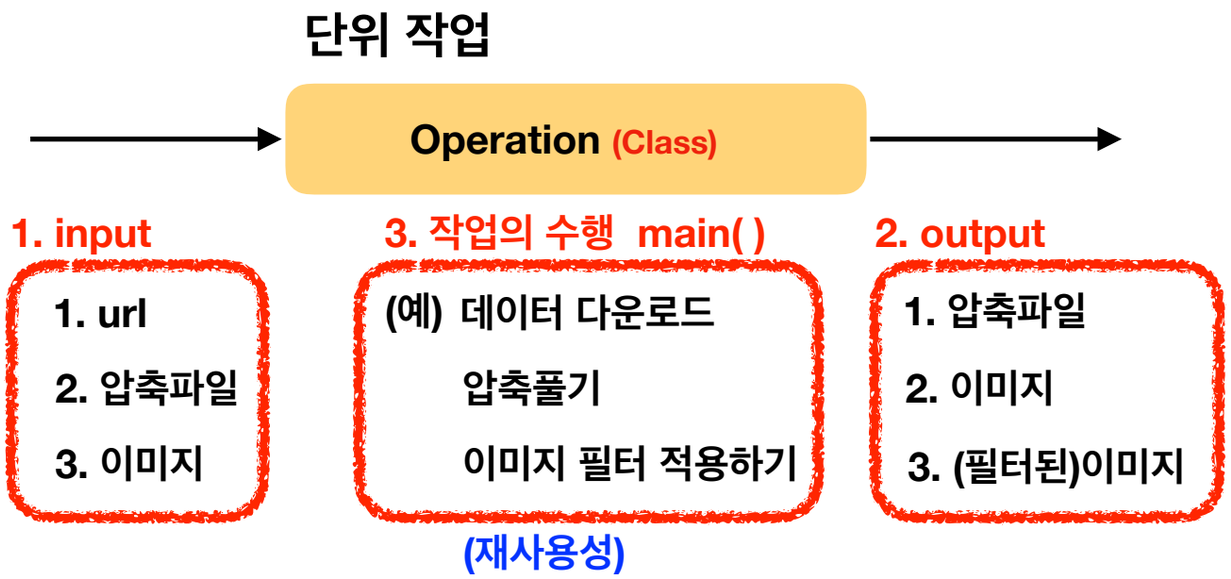
isReady	true
isExecuting	false
isCancelled	false
isFinished	false

isReady	true
isExecuting	true
isCancelled	false
isFinished	false

isReady	true
isExecuting	false
isCancelled	false/true
isFinished	true

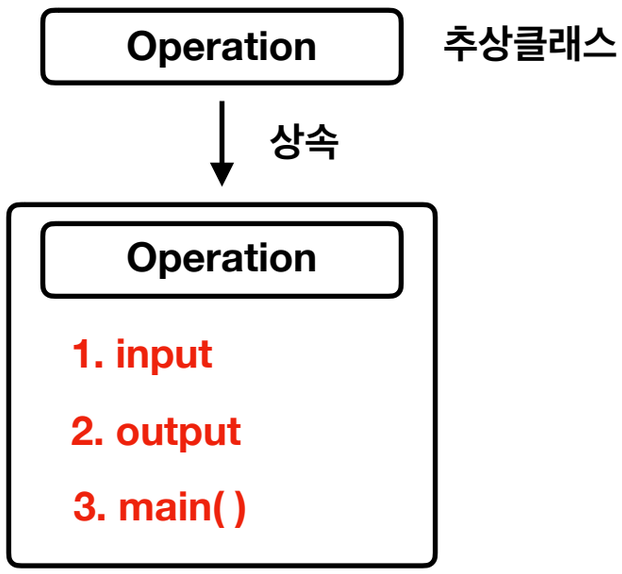
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Operation 기본개념



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Operation 기본개념



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
class SomeOperation: Operation {
    var inputImage: UIImage?
    var outputImage: UIImage?

    override func main( ) {
        정의할 메서드 내용
    }
}

let op = SomeOperation( )
```

실제 코드예시

2-1.Operation

2) OperationQueue (오퍼레이션큐)

Operation큐



(1) serial인지, concurrent인지?

몇개의 쓰레드를 사용할 것인지 구체적 설정 가능

디폴트 설정은 -1 (시스템이 알아서 여러개를 결정)
(defaultMaxConcurrentOperationCount = -1)

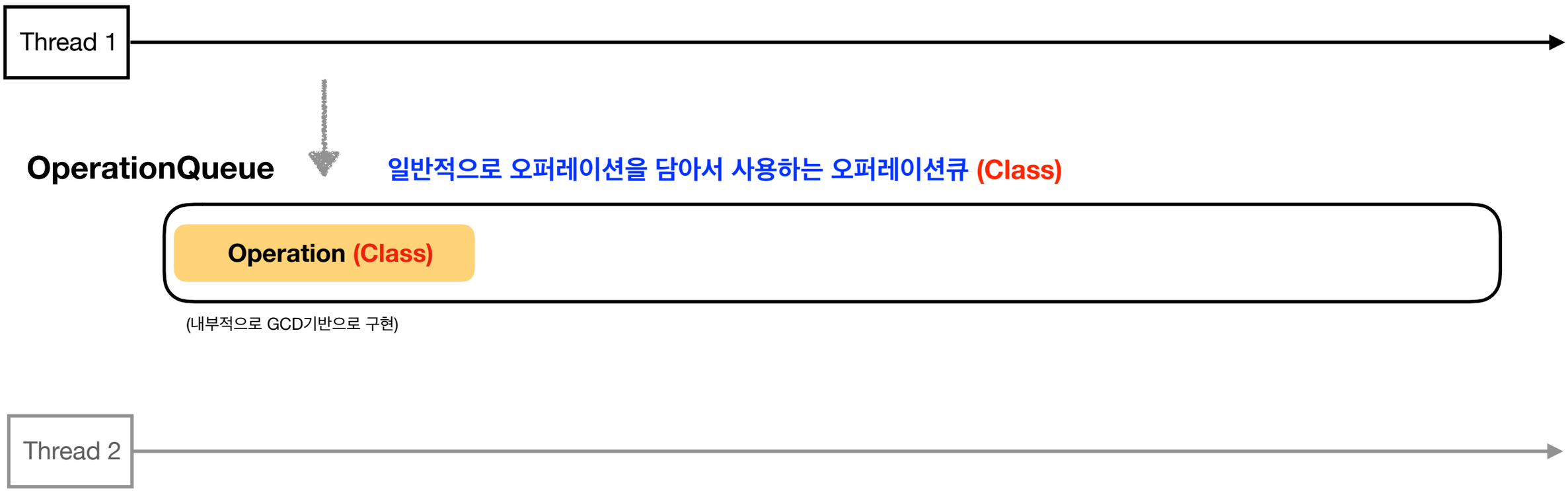
maxConcurrentOperationCount = 1 (시리얼)

maxConcurrentOperationCount = 2 (두개의 쓰레드 사용)

.
.
.

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Operation큐



OperationQueue 일반적으로 오퍼레이션을 담아서 사용하는 오퍼레이션큐 **(Class)**

Operation (Class)

(내부적으로 GCD기반으로 구현)

maxConcurrentOperationCount = 1 (시리얼)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Operation큐



(2) QoS(서비스 품질) 설정은?

1) 디폴트는 background. 구체적 설정 가능

queue.qualityOfService = .userInteractive

queue.qualityOfService = .userInitiated

queue.qualityOfService = .default

queue.qualityOfService = .utility

queue.qualityOfService = .background

2) 오퍼레이션 자체의 품질을 설정하면, Queue의 품질이 승격될 수있음

3) 다만, 기반이 되는(underlying) 디스패치큐까지
설정이 가능하기 때문에 underlying(디스패치)큐의 품질이 가장 우선적으로 적용됨

queue.underlyingQueue = .global( )

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Operation큐



(3) 오퍼레이션 추가 방법

클로저, 오퍼레이션, 오퍼레이션 배열(waitUntilFinised 아규먼트 존재)

(오퍼레이션큐는 오퍼레이션을 더하는 즉시, 스케줄링 하고 동시에 실행될 오퍼레이션의 갯수를 관리)

오퍼레이션을 오퍼레이션큐에 넣었을 때는 쓰레드에 배정되었을때 isExecuting 상태가 됨

(4) 오퍼레이션이 한번 실행되거나, 취소되면 오퍼레이션큐를 떠남(사라짐)

(5) 동기적으로 기다리는 메서드 존재 waitUntilAllOperationsAreFinished( )

(6) 기능: 일시중지-재개 가능

isSuspended = true / false 직접 설정 가능

(isSuspended = true 설정시 기존에 실행되던 오퍼레이션은 일단 계속 실행됨. 새로추가된 오퍼레이션은 isSuspended = false 될때까지 스케줄링 안됨)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
let someQueue = OperationQueue( )

someQueue.addOperation(oneOperation)
                    or
someQueue.addOperations([오퍼레이션 배열], waitUntilFinished: true)
```

실제 코드예시

2-2.OperationQueue

코드 그림표현

2-2.OperationQueue

Thread 1

filterQueue (오퍼레이션큐)

filterOp

Thread 2

filterOp

filterOp

Thread 3

filterOp

filterOp

Thread 4

filterOp

filterOp

Thread 5

filterOp

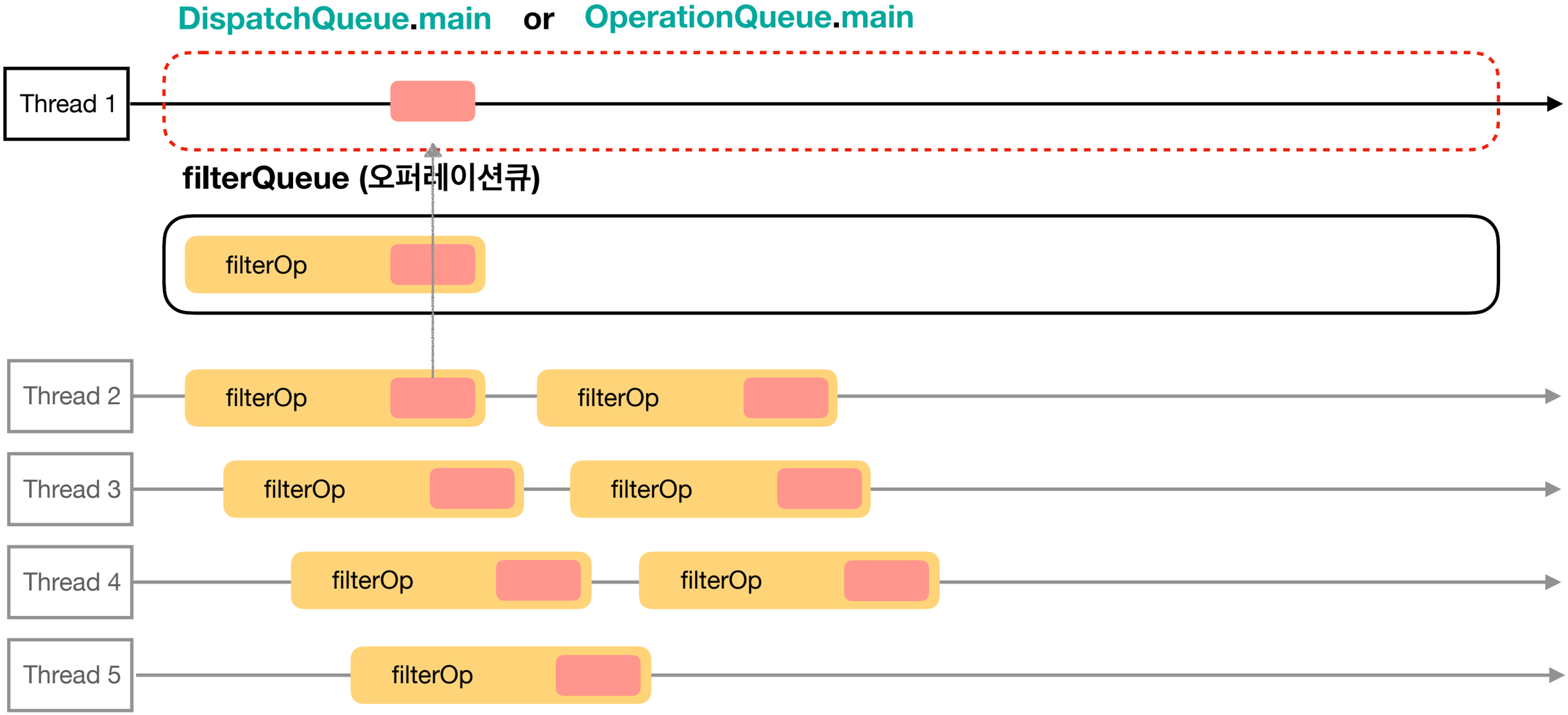
appendQueue (오퍼레이션큐)

Thread 6

메모리

filteredImages(배열)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com



만약에 오퍼레이션에서 완료하고 필요한 작업이 UI관련 작업이었다면?

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

참고. 8.압축해제 프로젝트

큐의 종류 정리

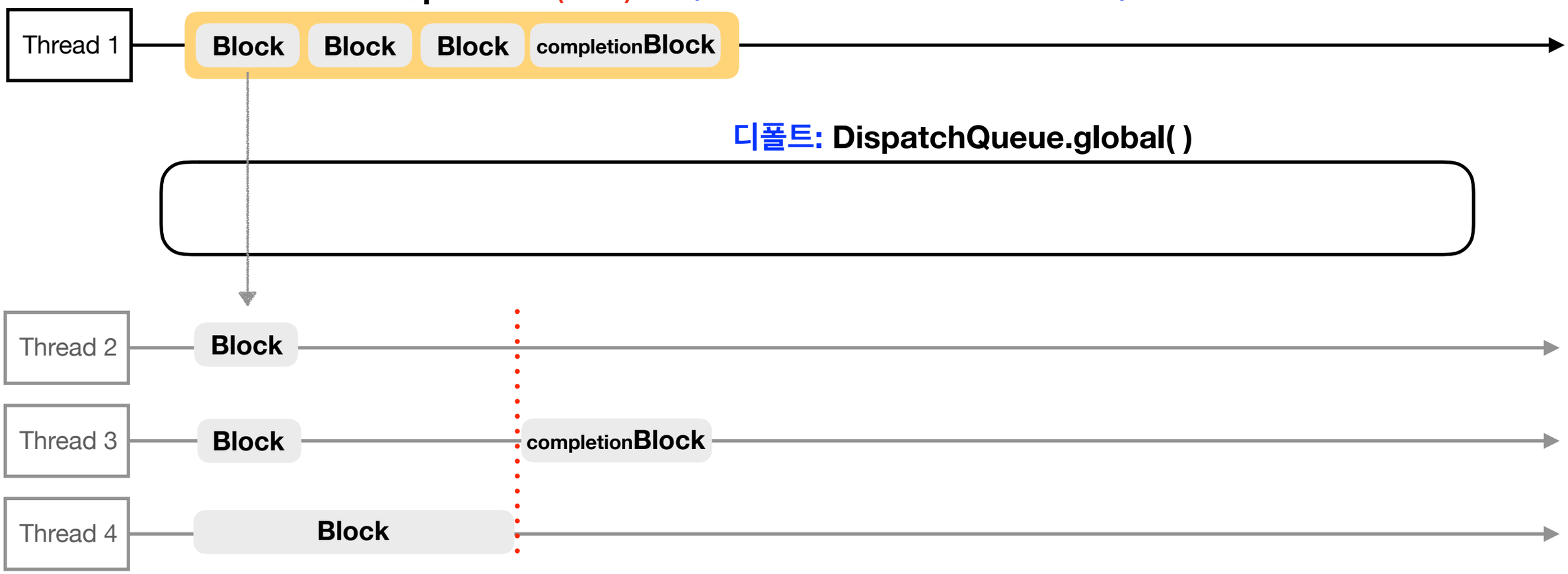
큐의 종류		생성 코드	특 징	직렬/동시
Dispatch Queue (GCD) (Class)	.main	DispatchQueue.main	메인큐 = 메인쓰레드 (1번쓰레드) (UI업데이트 내용 처리하는 큐)	Serial (직렬)
	.global( )	DispatchQueue.global(qos:)	6가지 Qos (작업에 따라 Qos상승가능) (시스템이 우선순위에 따라 더 많은 쓰레드를 배치하고, 배터리를 더 집중해서 사용하도록 함)	Concurrent (동시)
	custom	DispatchQueue(label: "…")	Qos추론 / Qos설정가능.	디폴트: Serial 둘다 가능 (attributes로 설정)
OperationQueue (Class) (내부적으로 GCD기반으로 구현)		let opQ = OperationQueue( )	디폴트: .background 기반(underlying) 디스패치큐에 영향 받음 (unspecified를 제외한 5가지)	디폴트: Concurrent 둘다 가능 (maxConcurrentOperationCount로 사용할 쓰레드 갯수 설정 가능)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

3) BlockOperation (블락오퍼레이션)

BlockOperation

BlockOperation (Class) (안에 block을 내장하고 있는 오퍼레이션)

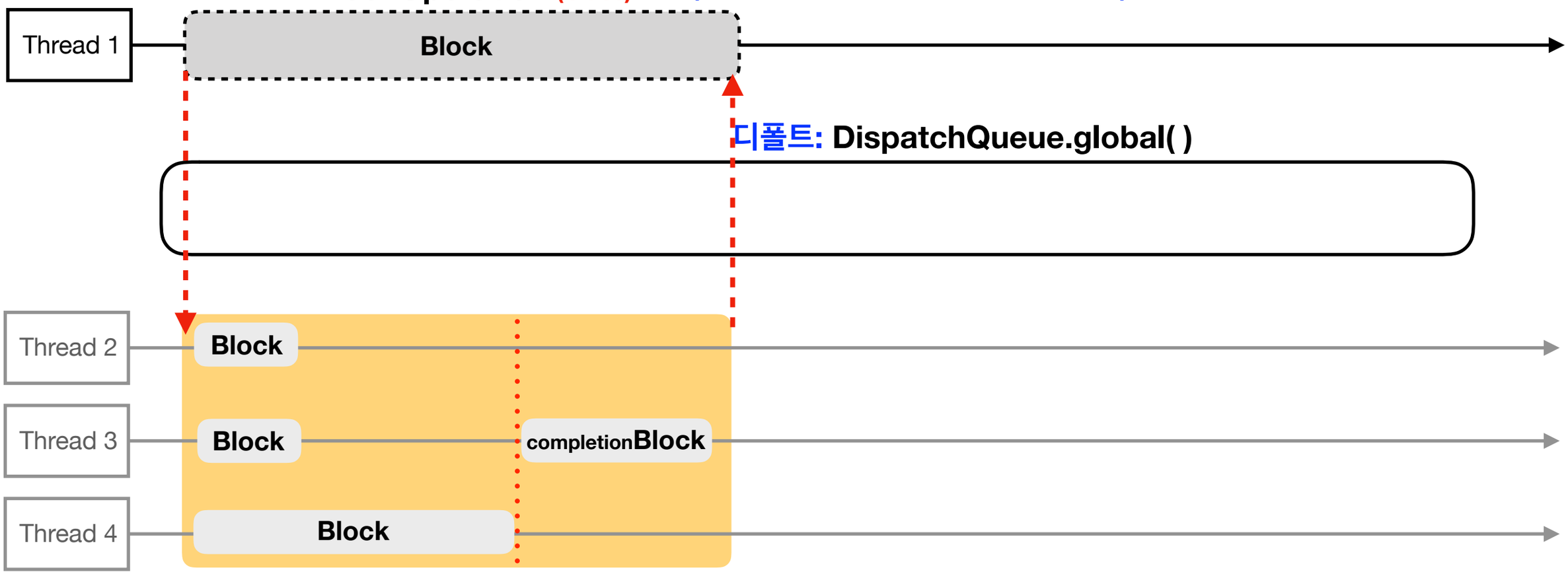


내부의 블락(클로저)들을 다른 쓰레드에서 일을 하도록 시킴

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

BlockOperation

BlockOperation (Class) (안에 block을 내장하고 있는 오퍼레이션)

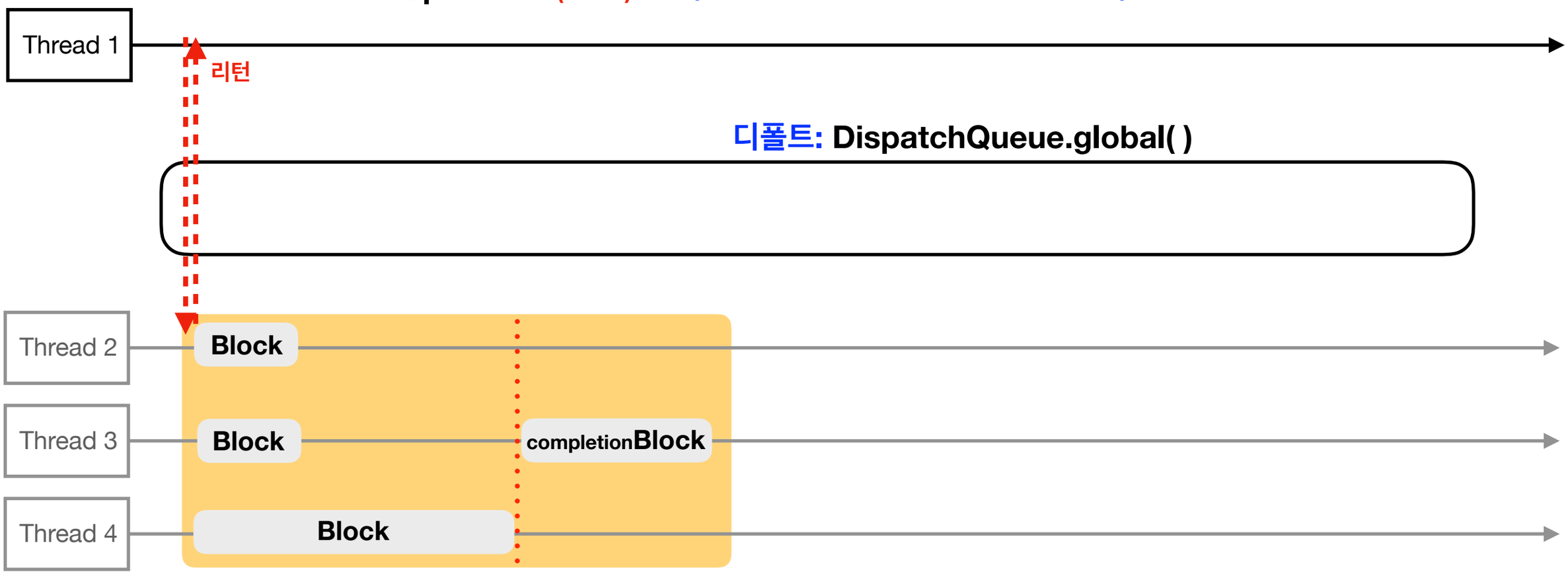


그렇지만, 기본설정은 동기적(sync)임

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

BlockOperation

BlockOperation (Class) (안에 block을 내장하고 있는 오퍼레이션)

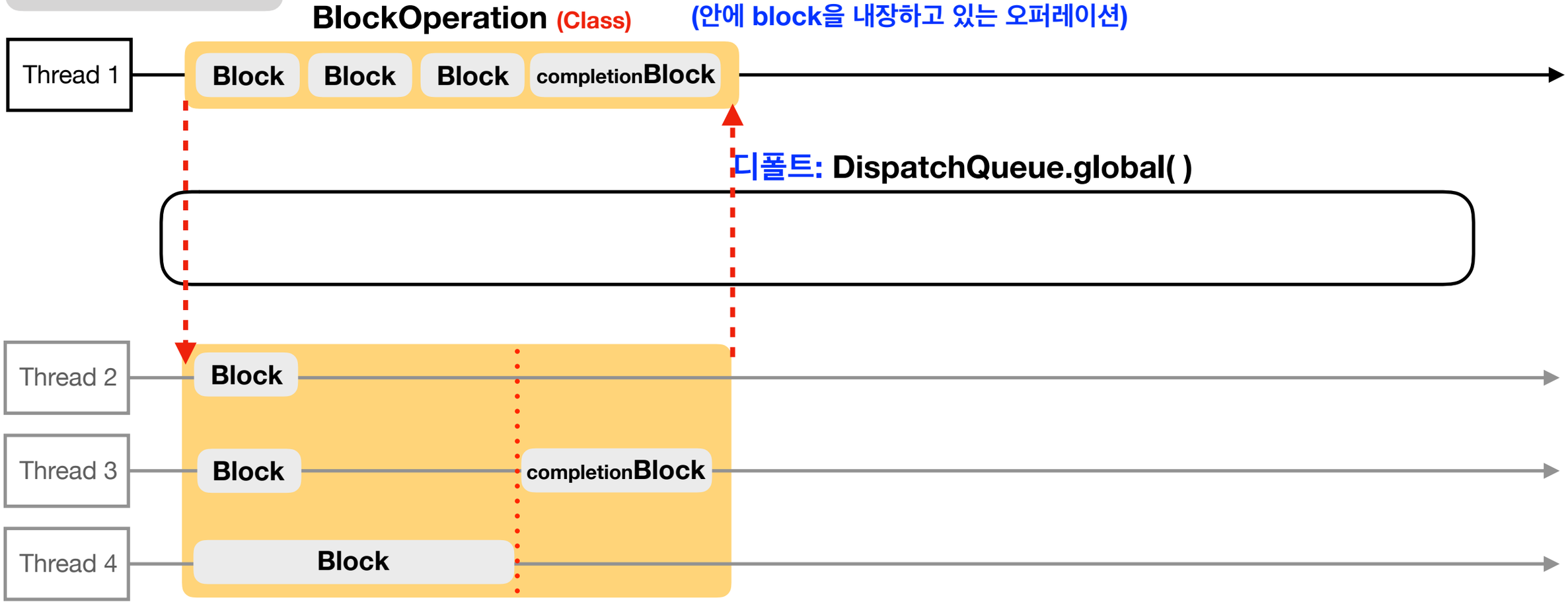


그렇지만, 기본설정은 동기적(sync)임

▶ 다른 큐로 보내서, 비동기적으로 실행하는 것도 가능

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

BlockOperation



▶ 디스패치 그룹과 유사하게 동작
(작업이 모두 종료 되었을때, 알려줌)

▶ 객체 지향의 래퍼 { }를 제공 ➡ 여러작업을 더할 수 있도록 만듦

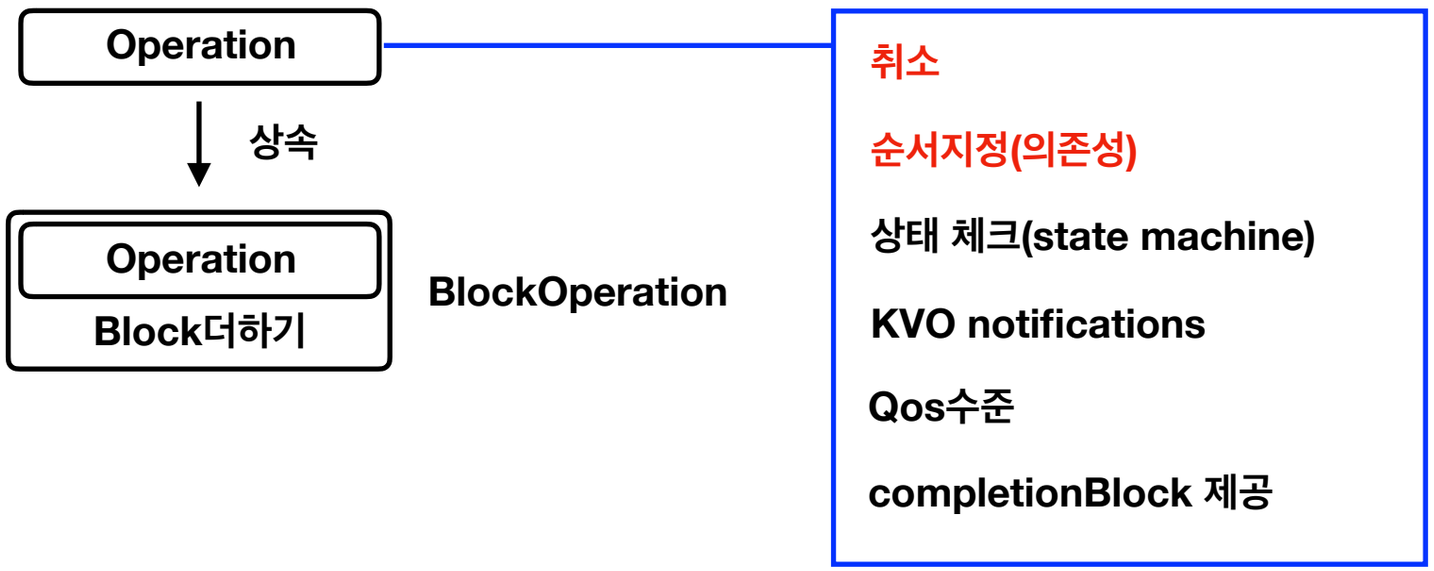
▶ 각 Block은 디폴트 글로벌 큐에서 동작 (Concurrent큐)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

BlockOperation

BlockOperation (Class)

Block | Block | Block | completionBlock



기능이 더 많은 디스패치 그룹

시리얼로 실행하기 원하면,
1) 시리얼 오퍼레이션큐로 보냄 2) 디펜던시(순서) 설정 (뒤에서 배움)

(주된 사용)
Operation을 많이 사용하는 앱에서, GCD와 같이 더 단순한
클로저가 필요한 경우에 주로 사용

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
let bOperation = BlockOperation( )

bOperation.addExecutionBlock {
        // 추가적으로 더할 수 있는 작업
}
```

or

```
let bOperation = BlockOperation {
        result = 2 + 3
        print(“print something”)
}
```

실제 코드예시

2-3.BlockOperation

실제 코드예시

GCD - 7-3.DispatchBarrier

3. AsyncOperation (비동기 오퍼레이션)

AsyncOperation

Operation

오퍼레이션 내에 비동기 함수를 내장할때

Thread 1

asyncMethod

Operation Queue

Thread 2

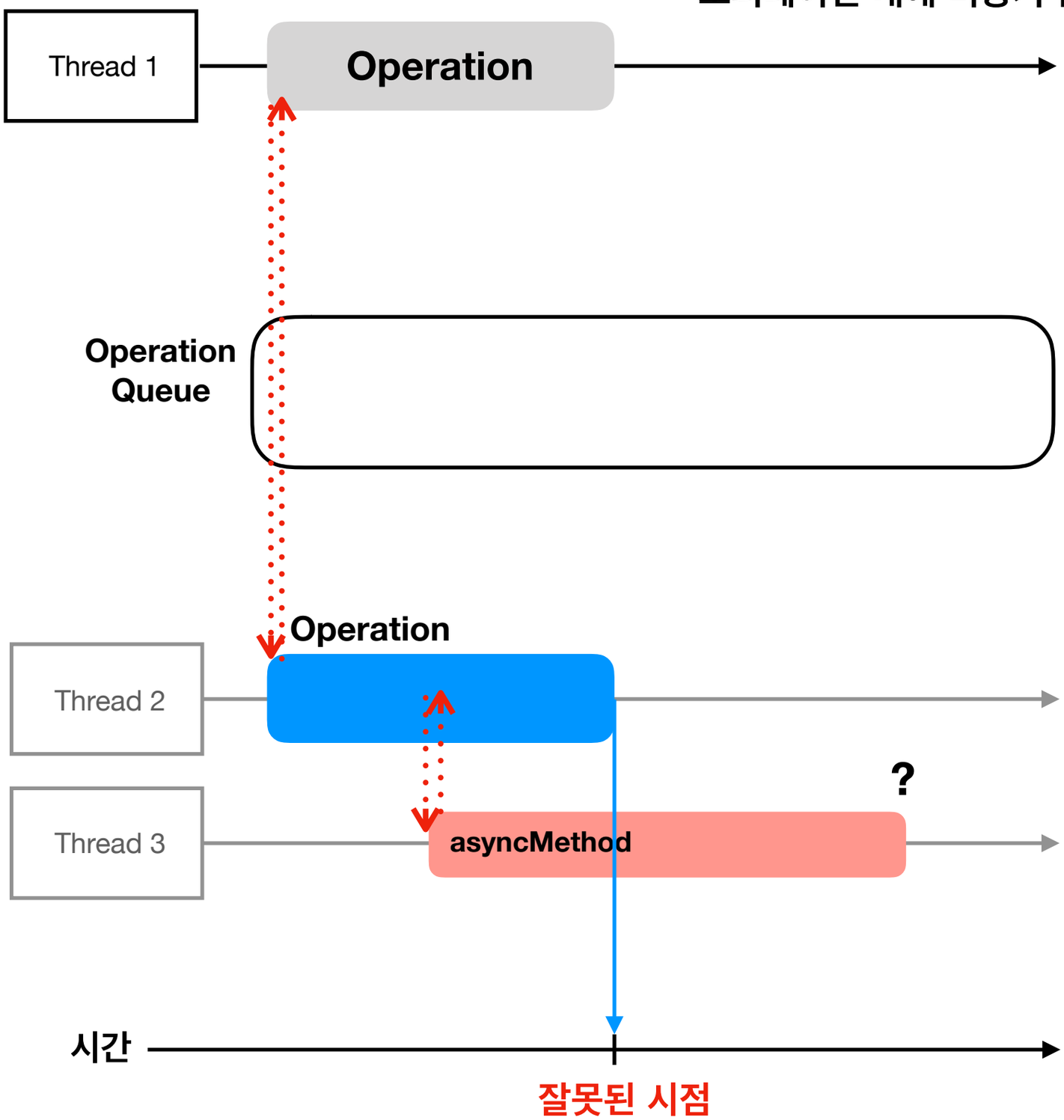
Thread 3

시간

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

AsyncOperation

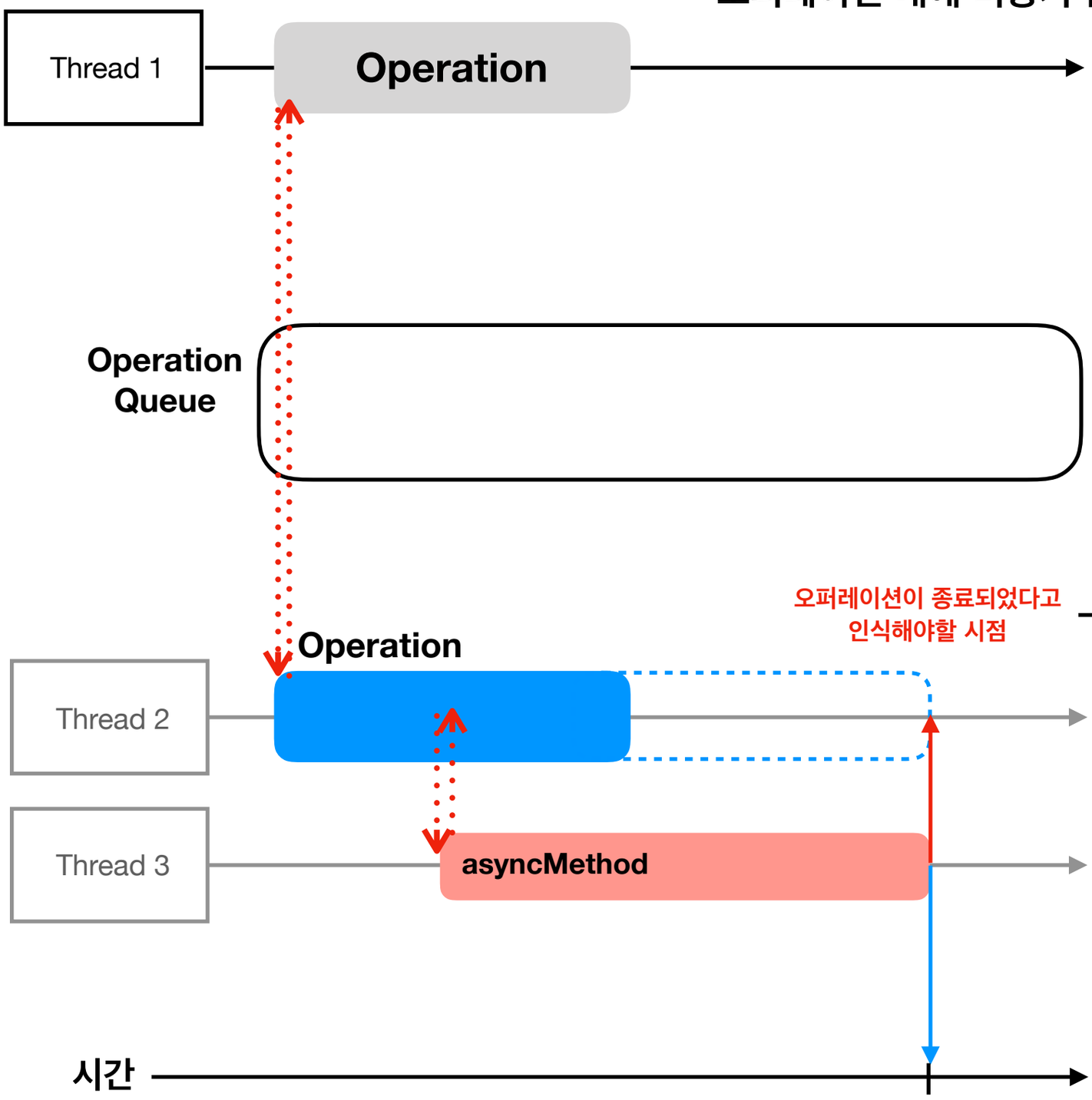
오퍼레이션 내에 비동기 함수를 내장할때



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

AsyncOperation

오퍼레이션 내에 비동기 함수를 내장할때



상태(state) 관리를 수동으로 해주어야 함

isReady (준비됨)

isExecuting (진행중)

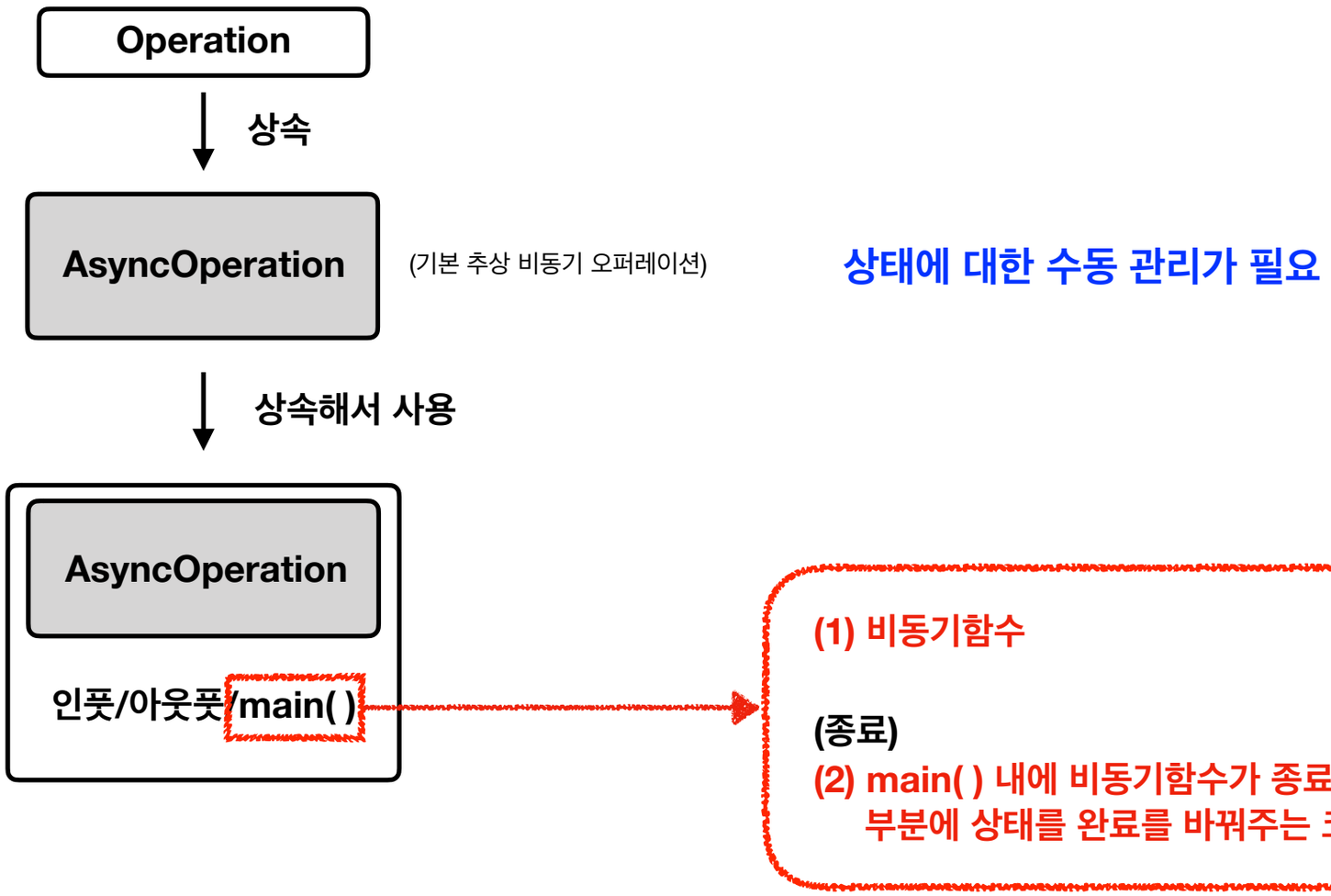
isFinished (종료됨)

(read-only 값이기에 직접 설정은 불가능)

+ notifications도 수동관리

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

AsyncOperation



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

3.AsyncOperation

AsyncOperation 3.AsyncOperation

enum

var state = (새롭게 생성한 변수)

- ready
- executing
- finished

→ state.keyPath =

- isReady
- isExecuting
- isFinished

ready → executing 경우의 예시

- willSet
  - willChangeValue(forKey: isExecuting) new밸류
  - willChangeValue(forKey: isReady)
- didSet
  - didChangeValue(forKey: isReady) old밸류
  - didChangeValue(forKey: isExecuting)

직접적인 설정이 불가능한 read-only 변수

- var isReady: super.isReady && state == .ready
  (객체 생성되어서, isReady되었는지 먼저 확인)
- var isExecuting: state == .executing
- var isFinished: state == .finished

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

AsyncOperation

비동기함수를 오퍼레이션으로 감싸는 이유?

순서 설정이 가능하기 때문에
(비동기 작업간의 순서 연결이 가능)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

AsyncOperation

```
loadData { (data) in
  loadImages(data, callback: { (images) in
    processImages(images, callback: { (result) in
      secondaryProcessing(result, callback: { (output) in
        DispatchQueue.main.async {
          print("This is your processed data:")
          for value in output {
            print(value)
          }
        }
      })
    })
  })
}
```

“pyramid of doom”을 피할 수 있음

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

4. 오퍼레이션의 주요 기능

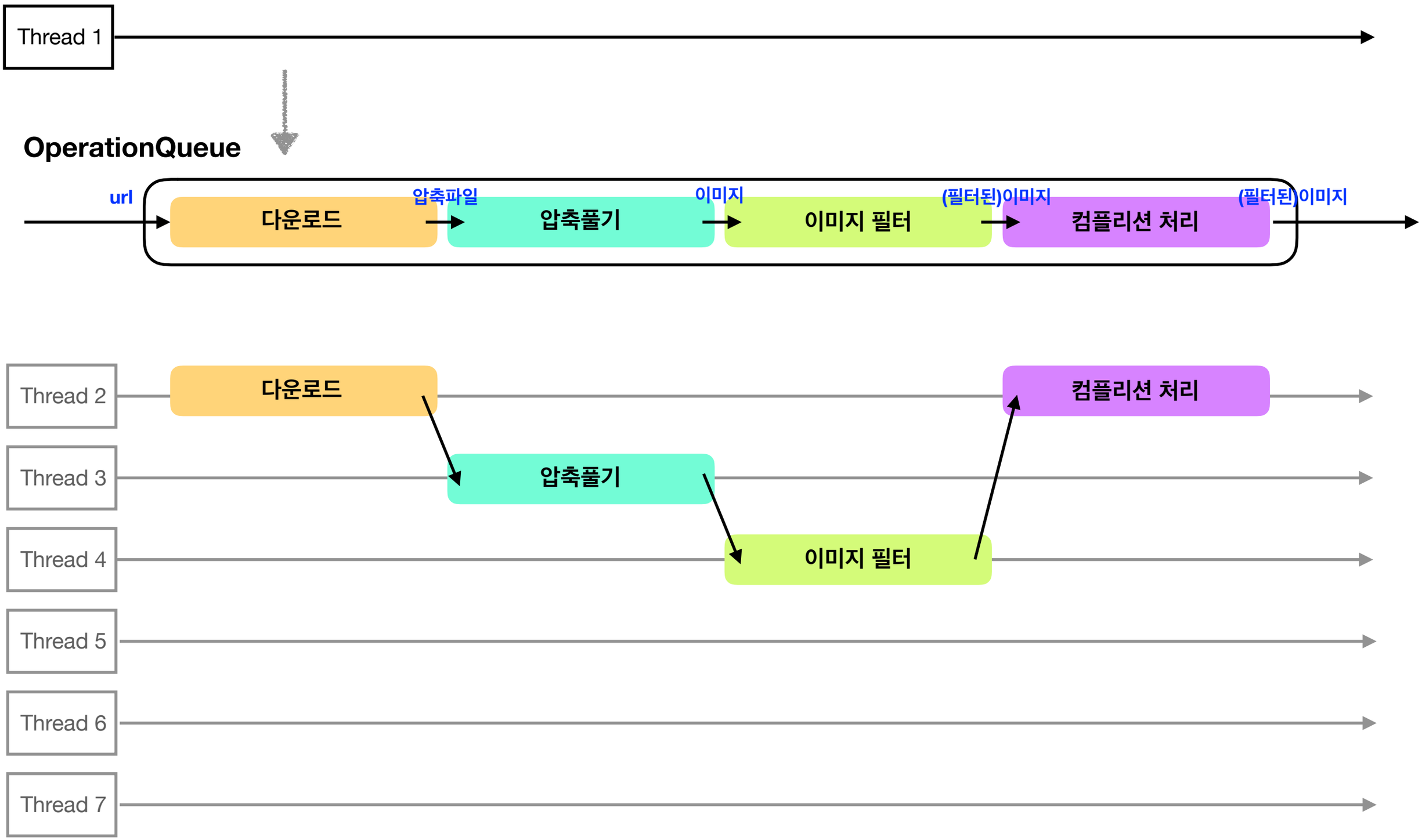
1) 순서 관리 (종속성 - Dependencies)

2) 오퍼레이션큐의 작업 취소

1) 순서 관리 (종속성 - Dependencies)

순서관리(종속성)

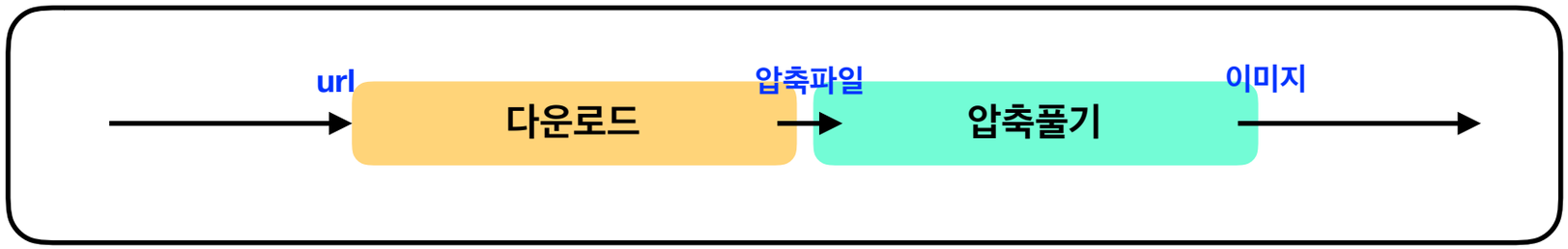
오퍼레이션큐는 어떻게 순서를 관리할까?



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

순서관리(종속성)

오퍼레이션큐는 어떻게 순서를 관리할까?



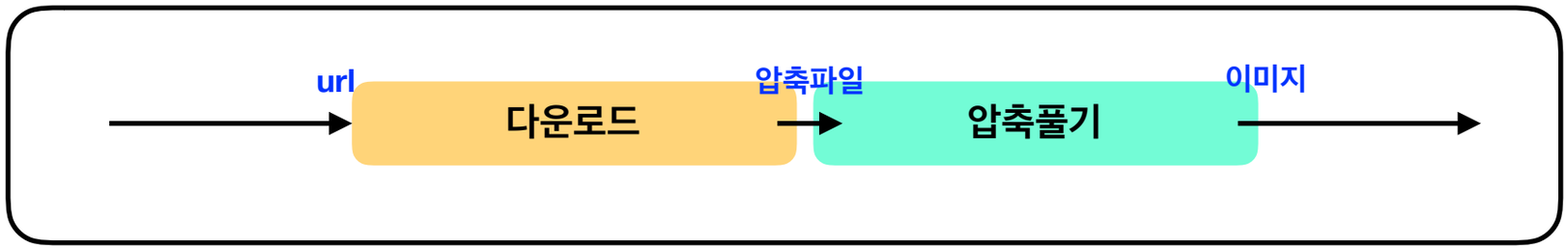


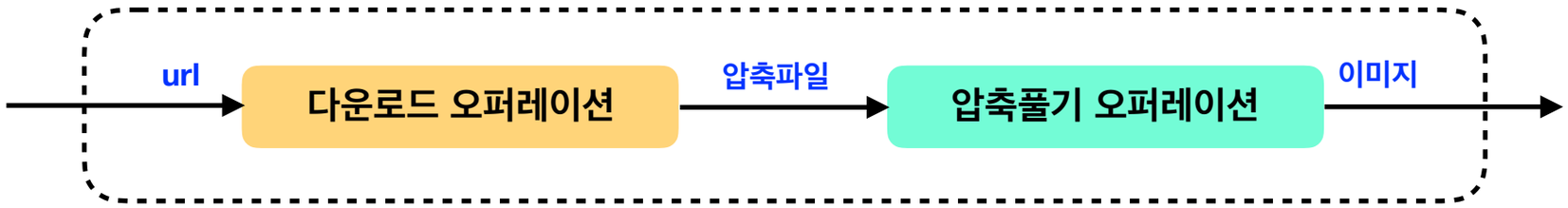
좋은 모듈 디자인이 아니다!

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

순서관리(종속성)

오퍼레이션큐는 어떻게 순서를 관리할까?



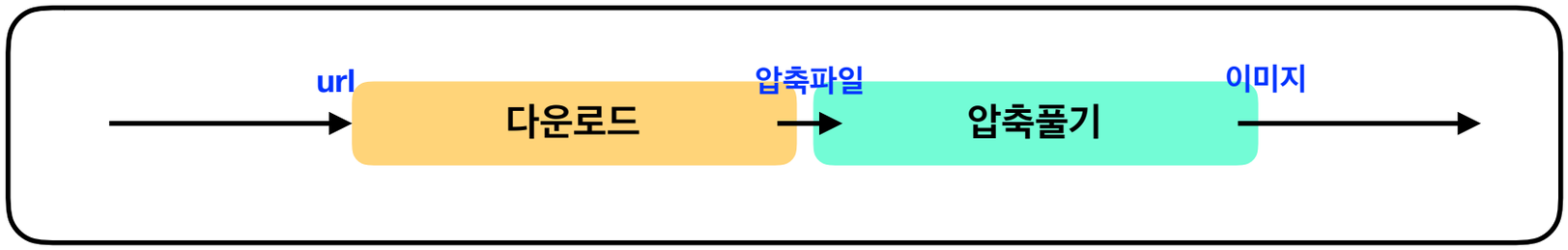


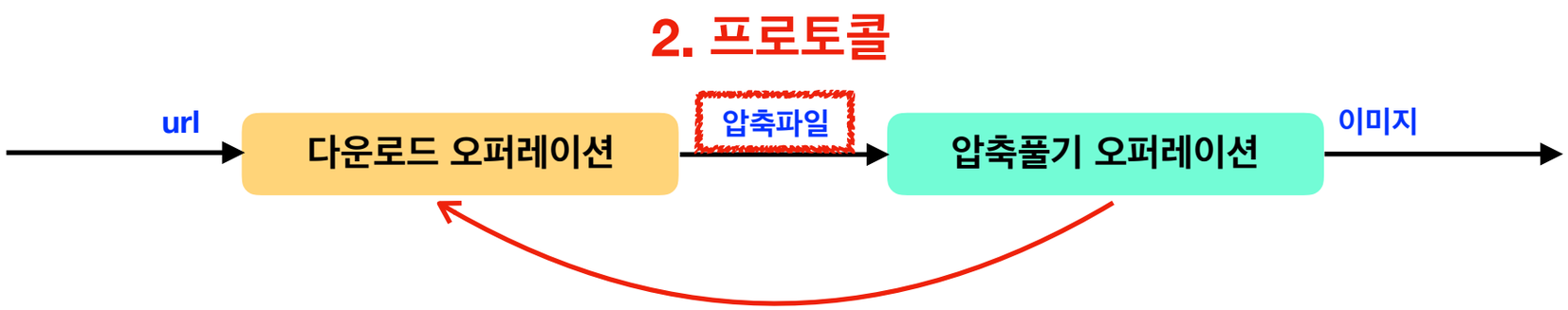
큐에서 첫번째 오퍼레이션이 끝나는 시점을 알고있고(관리하고),
그 작업이 끝나면 다음작업이 일을 시작하도록 한다.

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

순서관리(종속성)

오퍼레이션큐는 어떻게 순서를 관리할까?

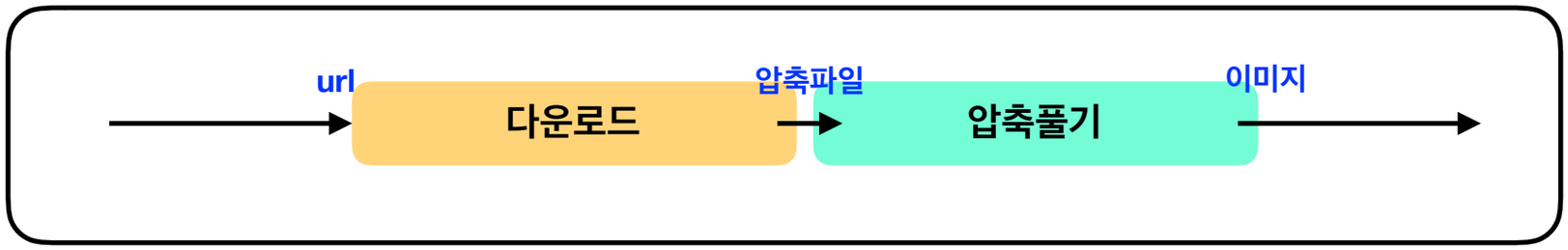




압축풀기 오퍼레이션이 다운로드 오퍼레이션에 의존한다.

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

오퍼레이션큐는 어떻게 순서를 관리할까?



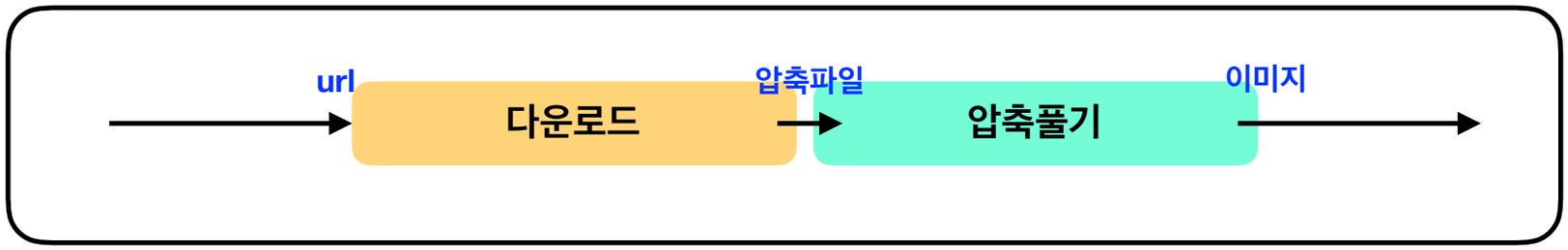
프로토콜 설정을 하지 않으면?

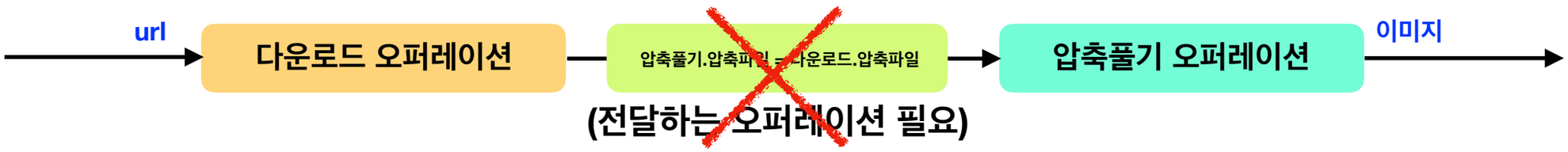


Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

순서관리(종속성)

오퍼레이션큐는 어떻게 순서를 관리할까?



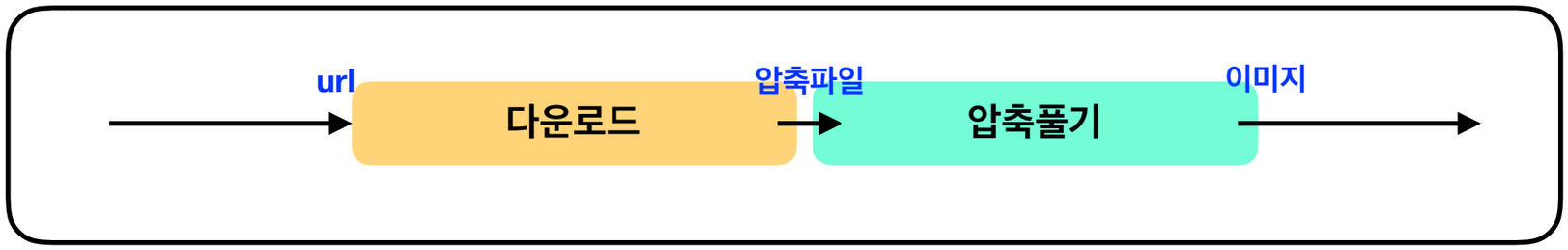


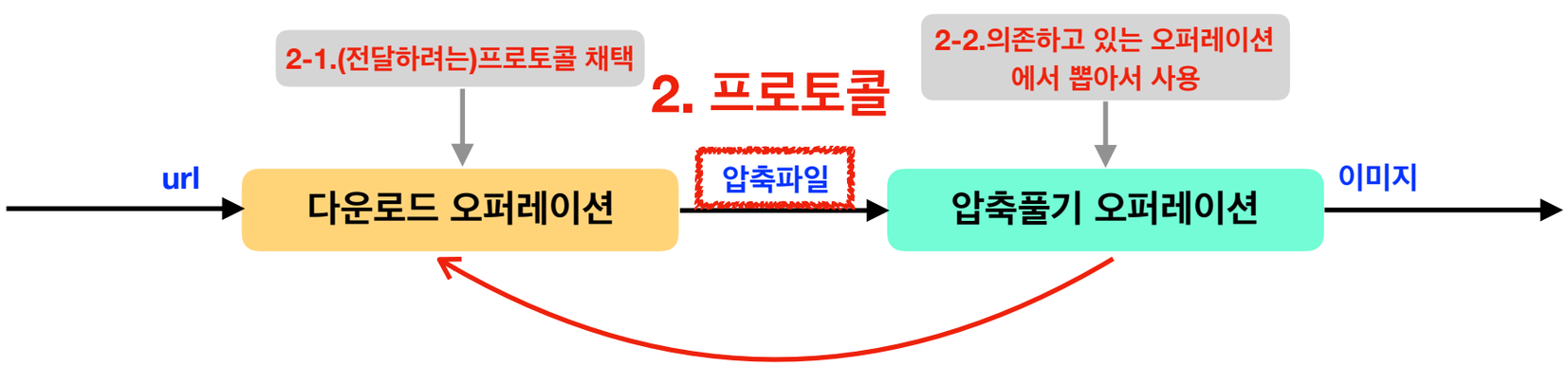
(전달하는 오퍼레이션 필요)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

순서관리(종속성)

오퍼레이션큐는 어떻게 순서를 관리할까?





압축풀기 오퍼레이션이 다운로드 오퍼레이션에 의존한다.

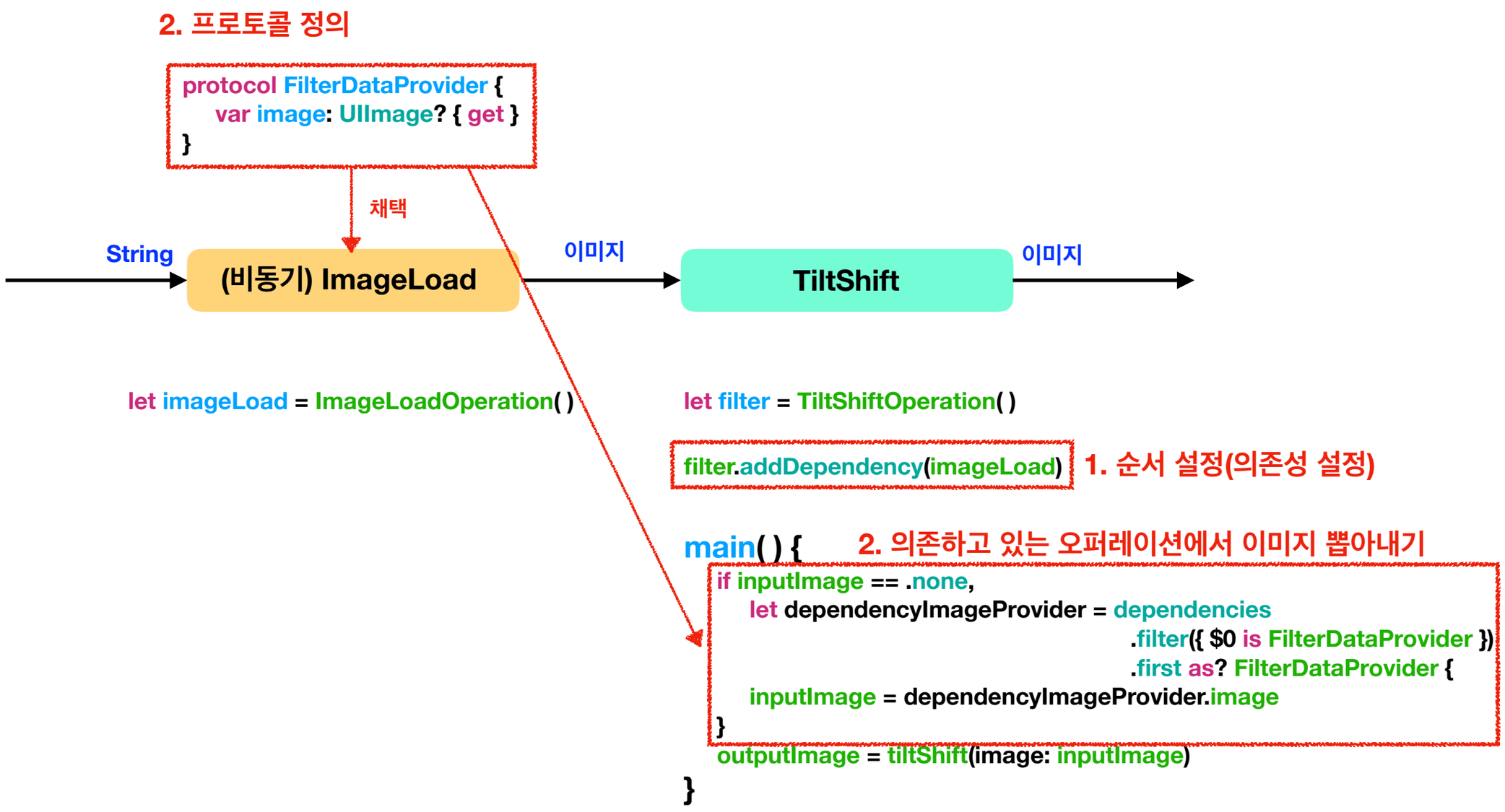
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

4-1.Dependency

코드 그림표현

4-1.Dependency



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

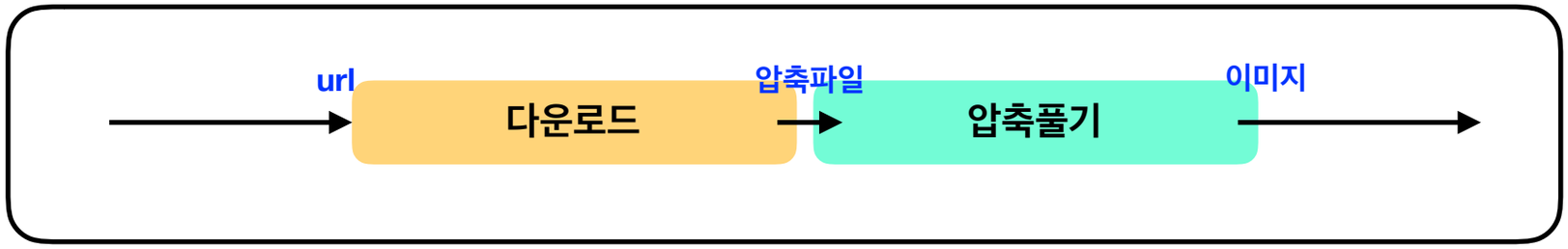
실제 코드예시

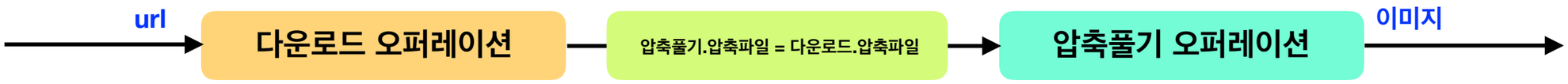
4-1.Dependency2

코드 그림표현

4-1.Dependency2 (프로토콜을 설정안한 경우의 예시)

오퍼레이션큐는 어떻게 순서를 관리할까?





(전달하는 오퍼레이션 필요)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Deadlock의 상황

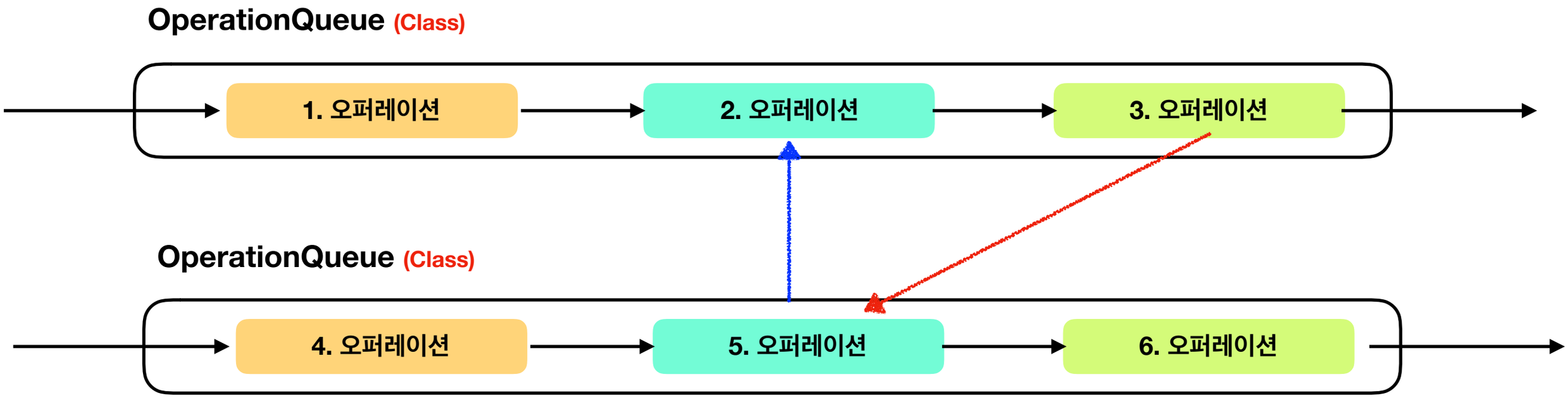


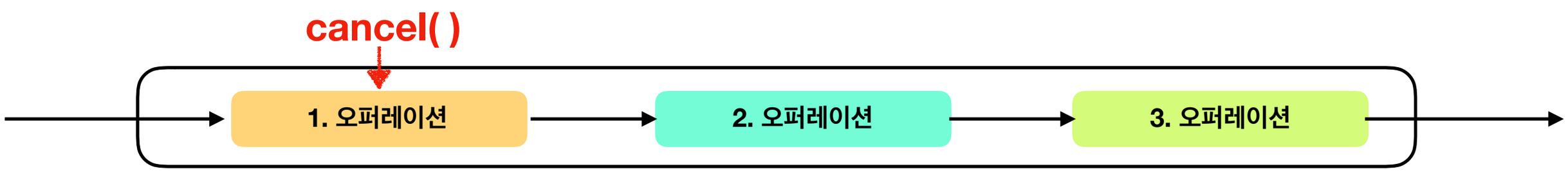
그림 참고: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

2) 오퍼레이션큐의 작업 취소

작업 취소

OperationQueue

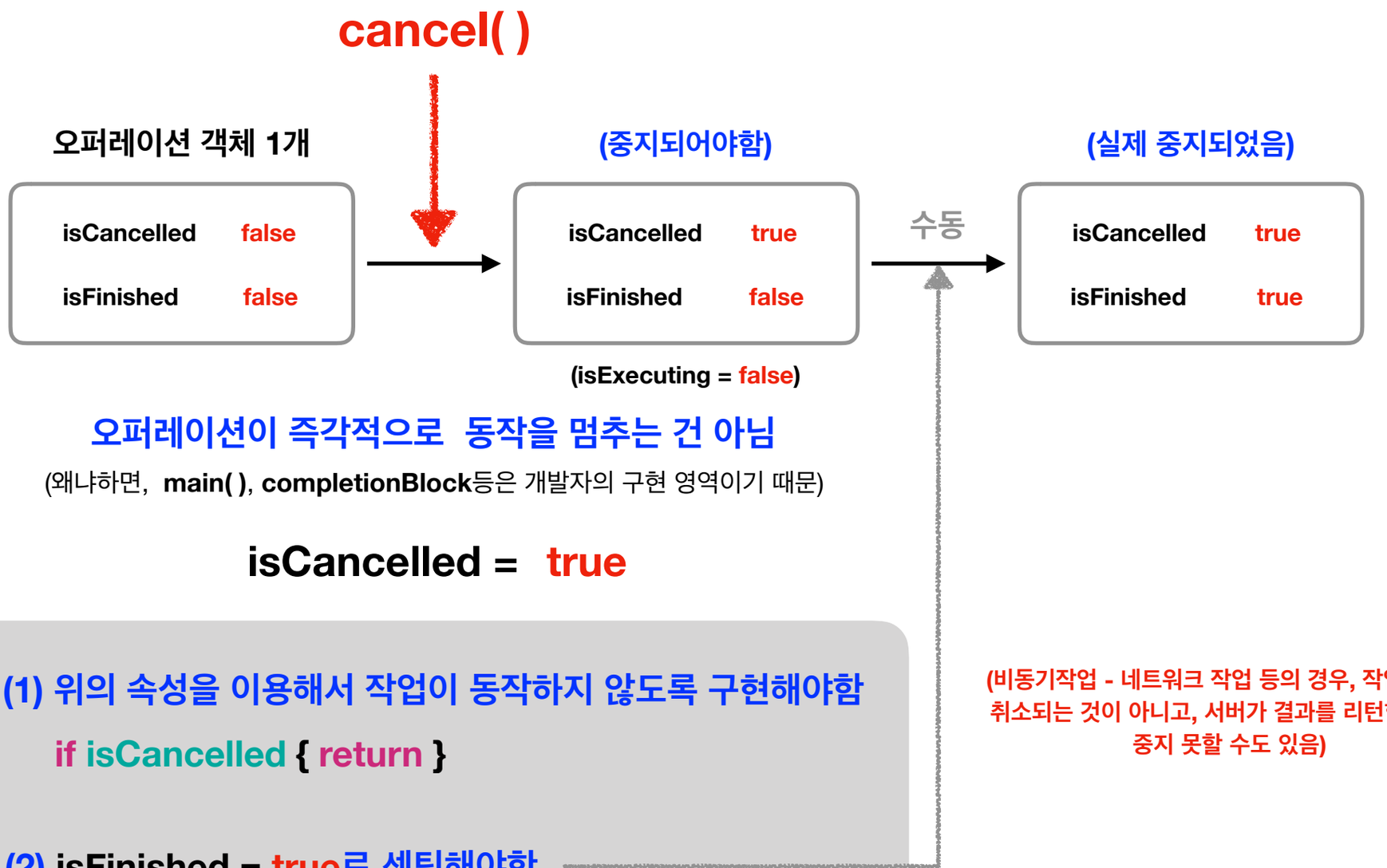


오퍼레이션이 오퍼레이션큐에 들어가면,
오퍼레이션큐가 관리하기 때문에 어떤 것도 할 수 없음이 원칙

그러나, 취소는 가능

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

작업 취소



오퍼레이션이 즉각적으로 동작을 멈추는 건 아님

(왜냐하면, **main( )**, **completionBlock**등은 개발자의 구현 영역이기 때문)

isCancelled = true

(1) 위의 속성을 이용해서 작업이 동작하지 않도록 구현해야함

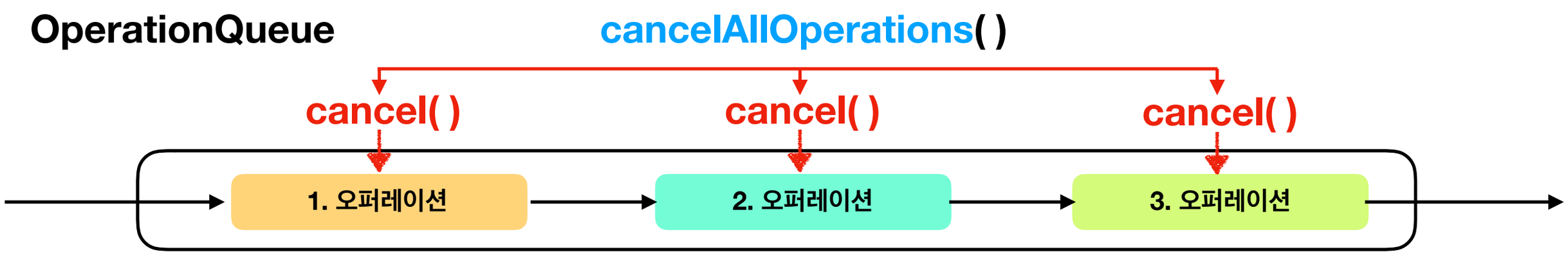
```
if isCancelled { return }
```

(2) isFinished = true로 셋팅해야함

(비동기작업 - 네트워크 작업 등의 경우, 작업이 바로 취소되는 것이 아니고, 서버가 결과를 리턴할때까지 중지 못할 수도 있음)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

작업 취소



isCancelled = true　　isCancelled = true　　isCancelled = true

자동적으로 작업들이 알아서 취소되는 것은 아님

이것을 사용해, 작업들이 취소 동작을 하도록 만들어줘야함

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

DispatchWorkItem의 작업 취소와 비교

구 분	DispatchWorkItem	OperationQueue
작업 객체의 취소	**cancel( )메서드** (isCancelled = true설정)	**cancel( )메서드** (isCancelled = true설정)
큐 내부에서	(작업 시작전이라면) 큐 내에서 작업이 제거됨	큐 내에서 isCancelled = true (isFinished 설정후 큐 내에서 작업이 제거됨)
작업 진행 막기	isCancelled이용	(1) isCancelled이용 (2) 상태관리위해 - isFinished = true
큐 전체의 작업 취소	**X**	✓ (cancelAllOperations( ))

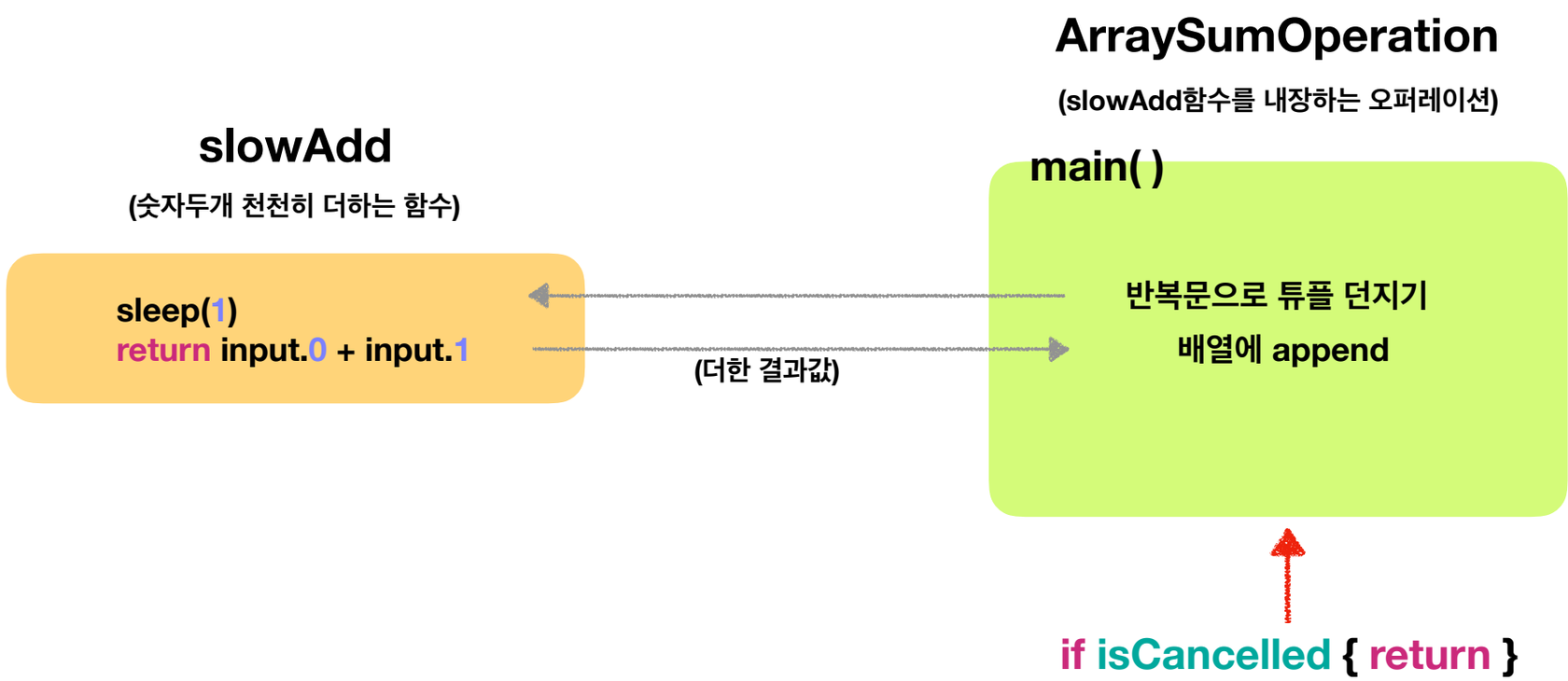
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

4-2.Cancellation

코드 그림표현

4-2.Cancellation (1번 케이스)



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드 그림표현

4-2.Cancellation (2번 케이스)

slowAddArray

(숫자두개 천천히 더해서, (중간에 계속 진행여부를 판단)
배열을 결과값을 가지는 함수)

results = [Int]

(계속진행여부 판단하는 클로저)

중간에 Double 값

return results

AnotherArraySumOperation

(slowAddArray함수를 내장하는 오퍼레이션)

main( )

반복문으로 튜플과 클로저 던지기

진행률 프린트(Double)

isCancelled = true / false

progress
- 배열 중간에 ← isCacelled = true
- 배열 끝까지 ← isCacelled = false

여부에 따라

끝까지 진행할 것인지를 판단하는 코드를 내장

if isCancelled { return }

~~if isCancelled { return }~~

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

4-2.Cancellation2

코드 그림표현 4-2.Cancellation2

class GroupAdd

var outputArray = 결과값을 저장하는 배열

오퍼레이션큐(queue)

2개의 쓰레드 사용

SumOperation SumOperation

Thread 2

SumOperation

Thread 3

SumOperation

오퍼레이션큐(appendQueue)

1개의 쓰레드 사용

Thread 4

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

참고 - 폴더 9.OperationCancellation

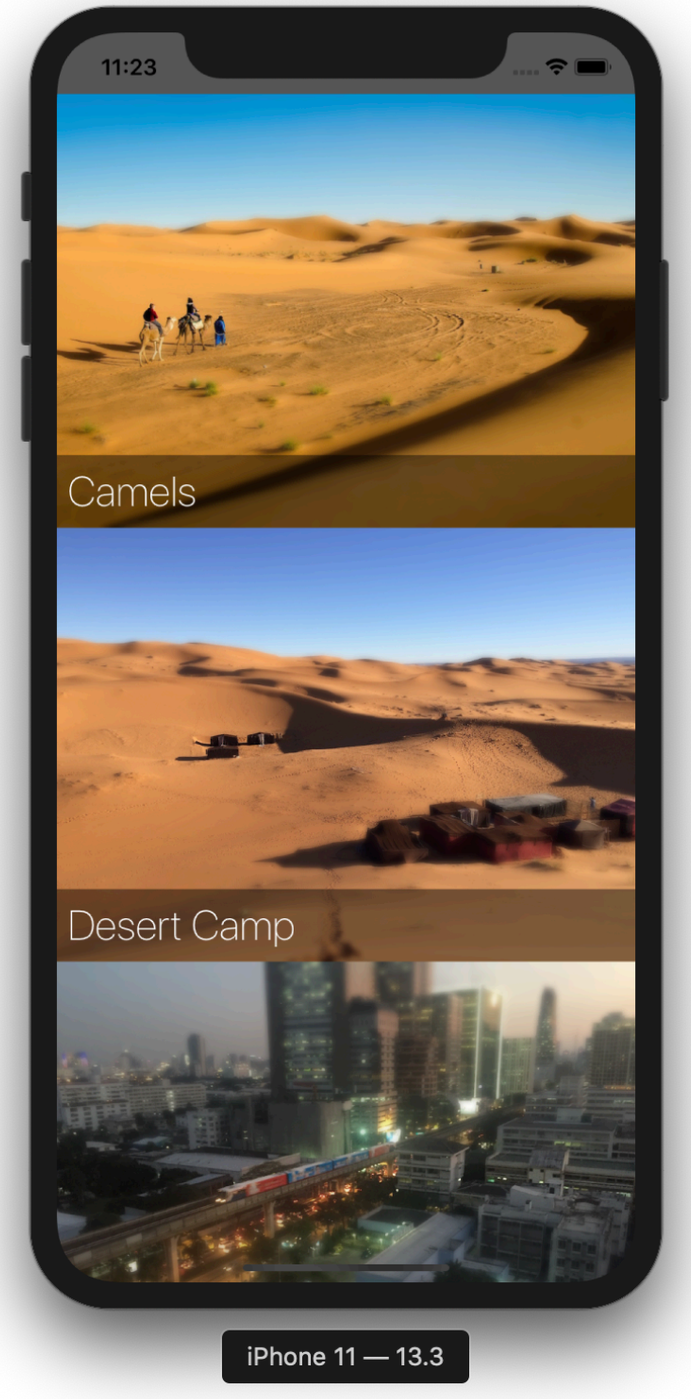
이미지 캐시처리 X
단순 취소 참고용 프로젝트

5. 오퍼레이션 프로젝트 살펴보기

실제 코드예시

폴더 10.Operation Project

Project



데이터소스

cellForRowAt
(실제 셀의 구성)

TiltShitImage (구조체) — imageName, title → **cell에 전달**

델리게이트

willDisplay
(셀 표시 하려고 할때의 동작)

didEndDisplaying
(셀 표시 끝났을 때의 동작)

TiltShiftImageProvider 클래스

다운로드해서, 압축풀고, 이미지필터링하는 작업

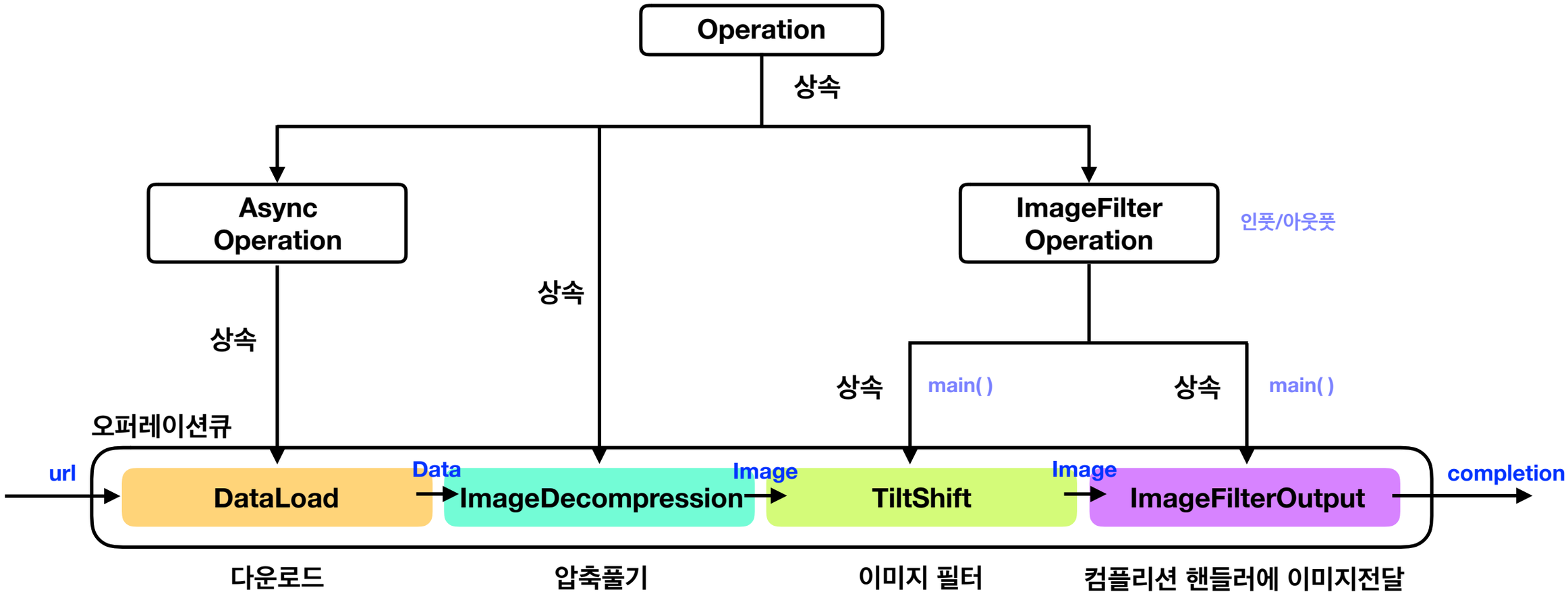
url → DataLoad → Data → ImageDecompression → Image → TiltShift → Image → ImageFilterOutput → completion

이미지 표시

TiltShiftImageProvider 클래스내에 들어있는 **오퍼레이션큐 취소**

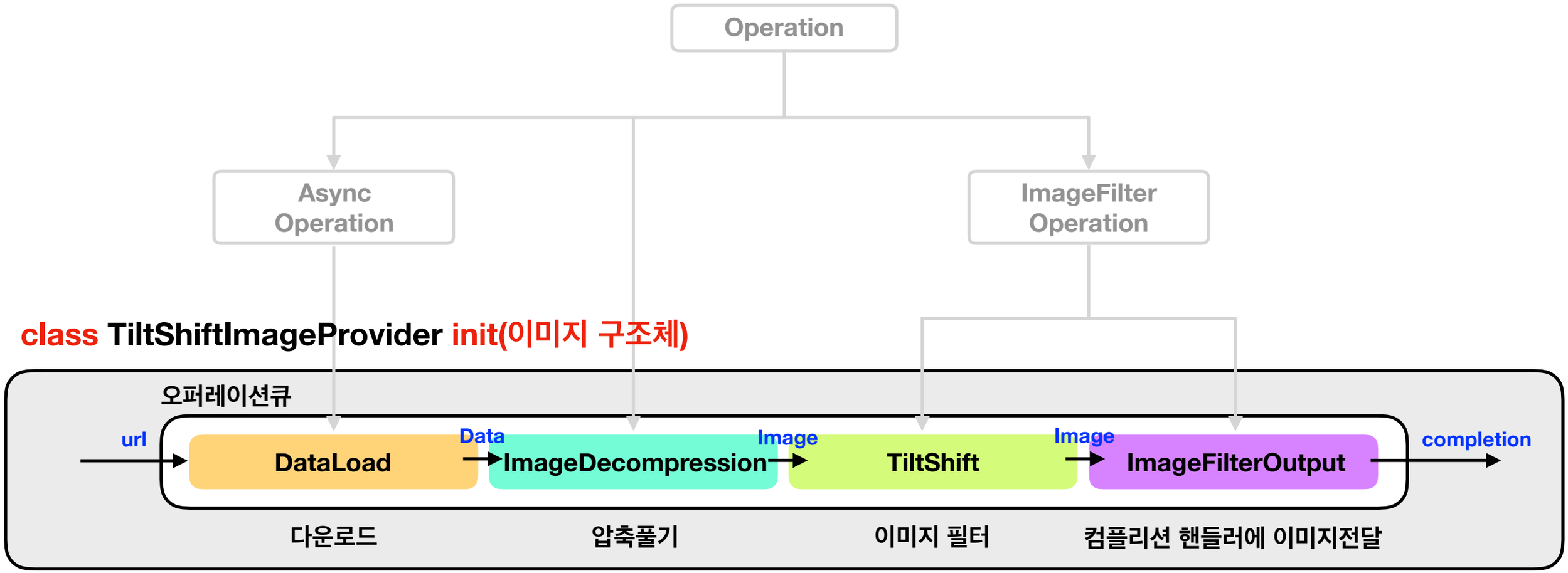
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Project 이미지 제공 오퍼레이션의 구성



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

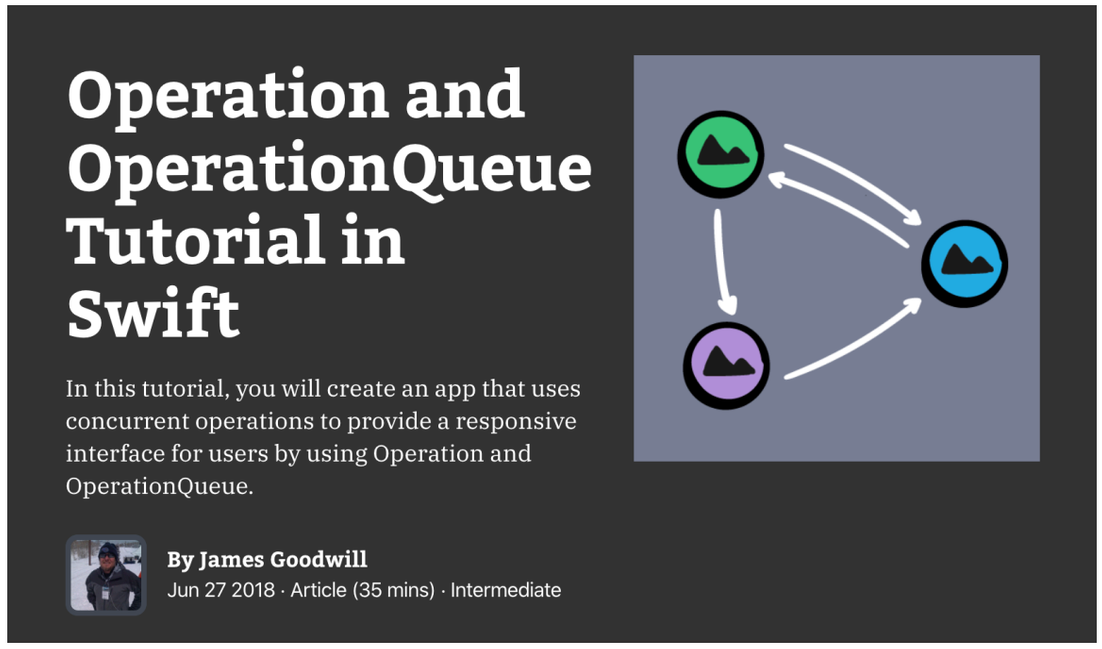
Project 이미지 제공 오퍼레이션의 구성



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

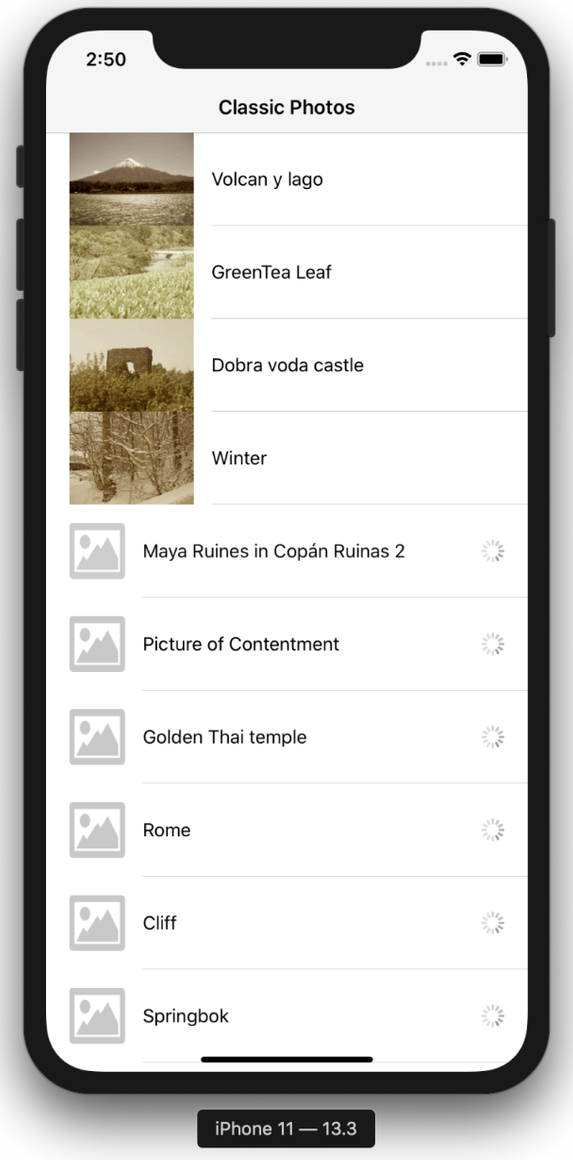
참고 프로젝트 **폴더. 오퍼레이션 프로젝트 관련 글 (관련 아티클 해석 pdf)**

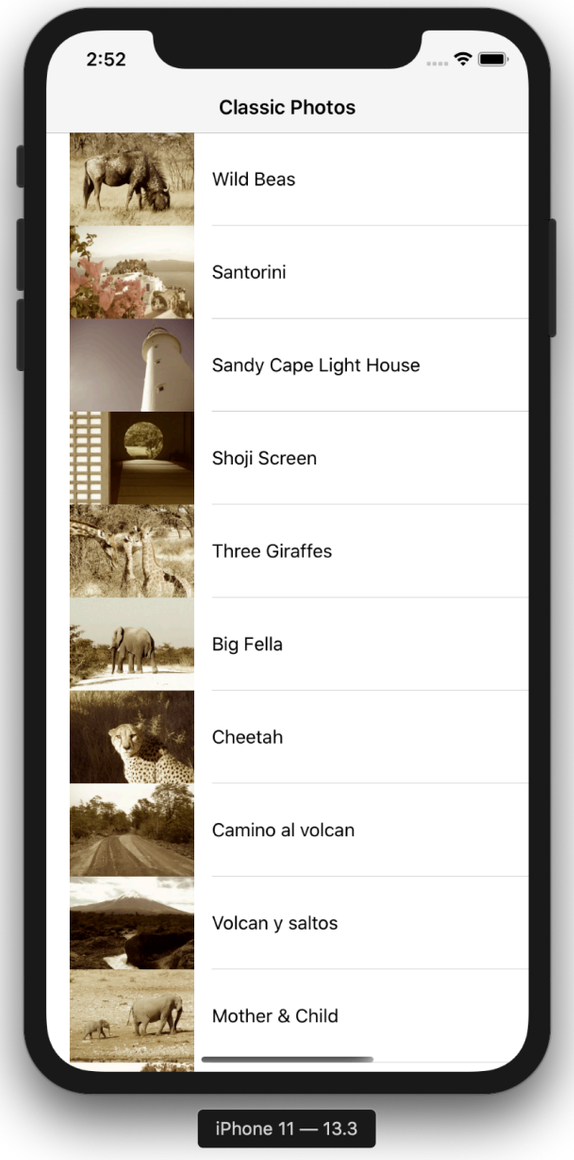
주소: https://www.raywenderlich.com/5293-operation-and-operationqueue-tutorial-in-swift



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

참고 프로젝트 폴더. 오퍼레이션 프로젝트 관련 글 (관련 아티클 해석 pdf)





스크롤뷰 델리게이트

스크롤 드래깅 시작 - Operation 일시중지

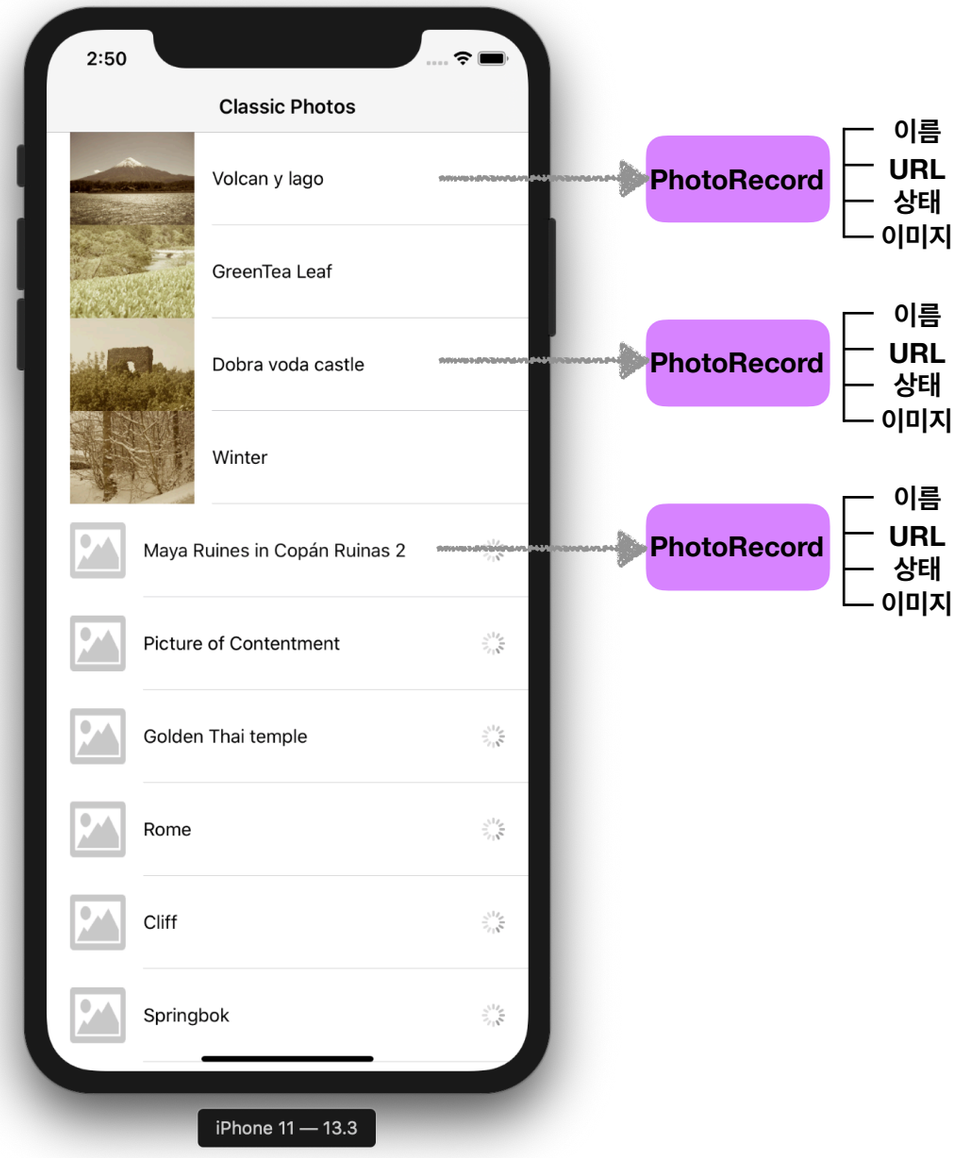
스크롤 드래깅 중지 - Operation 중지재개

스크롤 감속이 끝나면 - Operation 중지재개

프로젝트 다운로드 주소: https://www.raywenderlich.com/5293-operation-and-operationqueue-tutorial-in-swift

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

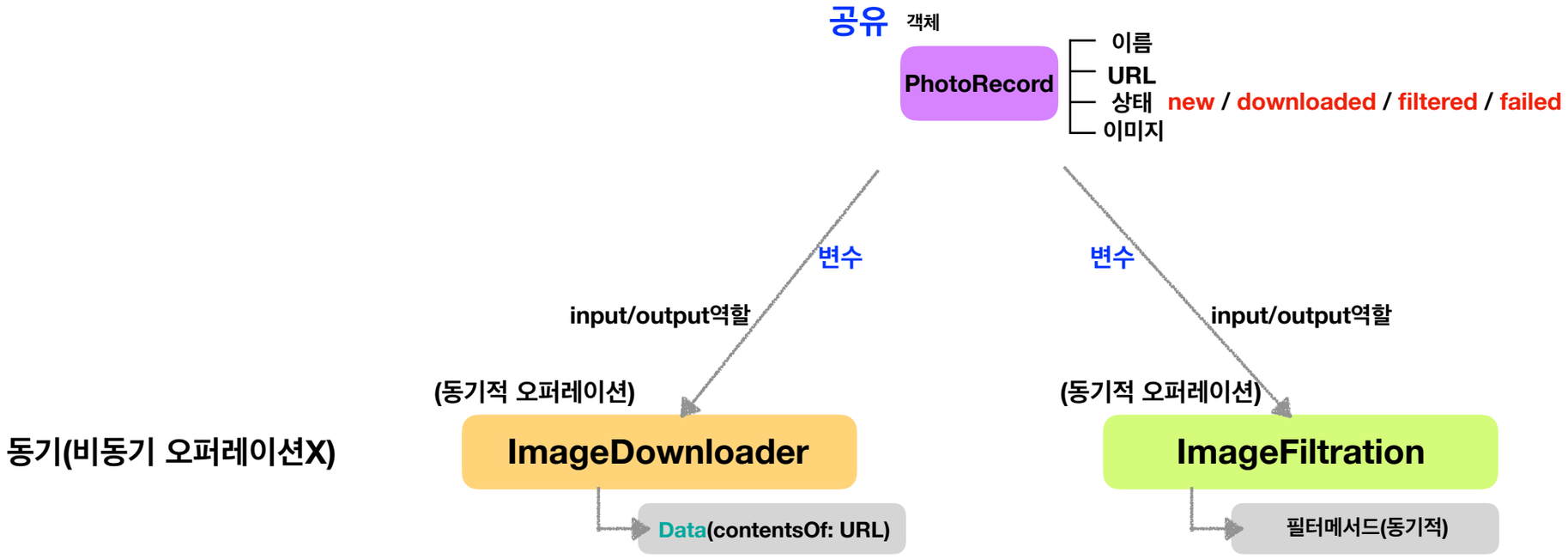
참고 프로젝트 **폴더. 오퍼레이션 프로젝트 관련 글 (관련 아티클 해석 pdf)**



프로젝트 다운로드 주소: https://www.raywenderlich.com/5293-operation-and-operationqueue-tutorial-in-swift

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

참고 프로젝트 폴더. 오퍼레이션 프로젝트 관련 글 (관련 아티클 해석 pdf)

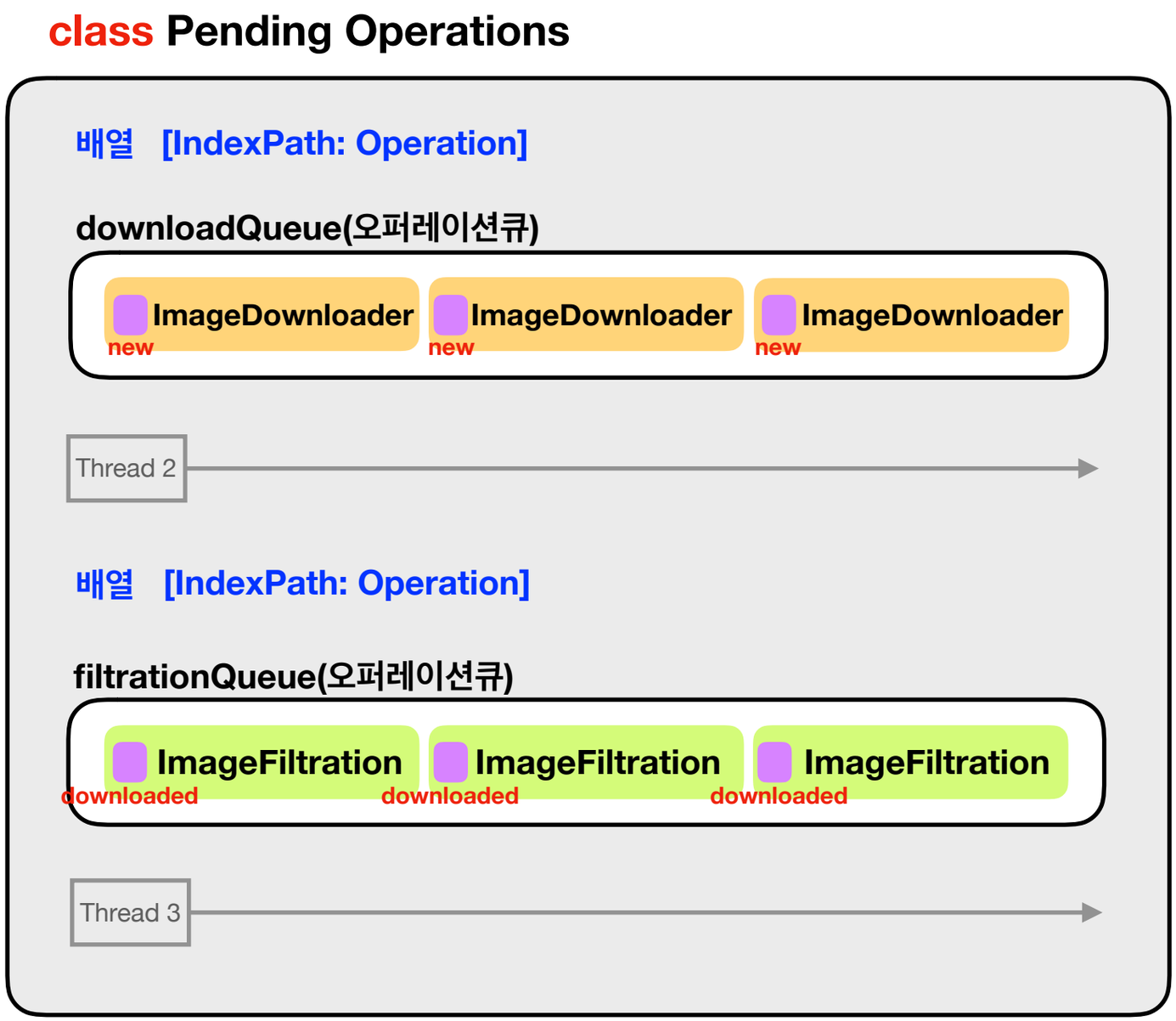


프로토콜을 이용한 순서설정을 하지는 않음

프로젝트 다운로드 주소: https://www.raywenderlich.com/5293-operation-and-operationqueue-tutorial-in-swift

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

참고 프로젝트 폴더. 오퍼레이션 프로젝트 관련 글 (관련 아티클 해석 pdf)



프로젝트 다운로드 주소: https://www.raywenderlich.com/5293-operation-and-operationqueue-tutorial-in-swift

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

6. 총정리 - GCD / Operation의 비교와 사용

Operation

1. Operation에 앞서

1) GCD와 Operation의 차이

2) 언제 Operation(오퍼레이션)이 필요할까?

3) 들어가기에 앞서

2. Operation / OperationQueue

1) 오퍼레이션(Operation)

2) OperationQueue (오퍼레이션큐)

3) BlockOperation (블락오퍼레이션)

3. AsyncOperation (비동기 오퍼레이션)

4. 오퍼레이션큐의 주요 기능

1) 순서 관리 (종속성 - Dependencies)

2) 오퍼레이션큐의 작업 취소

5. 오퍼레이션 프로젝트 살펴보기

6. 정리 - GCD / Operation의 비교와 사용

GCD

1. GCD/Operation에 앞서

1) 왜 동시성(Concurrent) 프로그래밍이 필요할까?

2) 간단한 GCD/Operation 소개

3) Synchronous(동기) VS Asynchronous(비동기)

4) Serial(직렬) VS Concurrent(동시)

2. 디스패치큐(GCD)의 종류와 특성

1) 메인 큐

2) 글로벌큐와 Qos의 이해

3) 프라이빗(커스텀) 큐

3. 디스패치큐(GCD) 사용시 주의해야할 사항

1) 반드시 메인큐에서 처리해야하는 작업

2) sync 메서드에 대한 주의사항

3) weak, strong캡처 주의

4) 컴플리션핸들러의 존재이유

5) 동기적 함수를 비동기함수 처럼 만드는 방법

4. 디스패치 그룹

1) 디스패치 그룹의 개념

2) 디스패치 그룹의 사용
- 디스패치 그룹과 동기함수
- 디스패치 그룹과 비동기함수

3) (참고) Dispatch WorkItem

4) (심화) Semaphore의 이해

5. (참고) GCD 프로젝트 살펴보기

6. (심화) 동시성과 관련된 문제

1) Race Condition (경쟁상황)

2) Deadlocks (교착상태)

3) Priority Inversion (우선 순위의 뒤바뀜)

7. (심화) Thread-safe한 코드의 구현과 방법

1) TSan 사용법

2) 시리얼큐와 sync

3) 디스패치 배리어(Dispatch Barrier)

4) 객체 설계 시 주의할 점

8. (심화) Lazy var와 관련된 이슈들

GCD / OperationQueue

GCD

- 간단한 일(커뮤니케이션의 양)
- 함수를 사용하는 작업 (메서드 위주)

/

Operation

- 복잡한 일(커뮤니케이션의 양)
- 데이터와 기능을 캡슐화한 객체

기반

(+ 여러가지 기능)

취소 / 순서지정 / 일시중지 (상태추적)

프로젝트의 효율성, 사례 적합성 등

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

GCD / OperationQueue

구 분	GCD	OperationQueue
주요 사용 목적	다른 쓰레드에서 비동기적 처리 (네트워크 처리 등) →	여러가지 추가적 기능 활용 (내부적으로 GCD기반 처리)
Task의 정의	메서드/코드 덩어리 (디스패치 워크아이템은 Class)	데이터와 기능 캡슐화 **(재활용 가능)** (Class)
작 업 취 소	**X** (디스패치 워크아이템만 가능)	✓
순서설정(의존성)	**X** (시리얼큐로 가능 - 데이터는 직접전달) (디스패치워크아이템은 바로 다음작업은 설정 가능)	✓
작업 일시 중지	**X**	✓
상 태 추 적	**X**	✓
장벽 작업(Barrier)	✓	(순서화 가능하기때문에 불필요)
그룹 작업	**디스패치 그룹** (여러작업 묶음의 끝나는 시점을 알기 위함)	**블락오퍼레이션으로 가능**
언 제 사 용 ?	**프로젝트의 장기 지속 가능성, 효율성 및 특정 사용 사례에 적합한 기술을 사용할 필요**	

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com